

Question 1

Each XML file in a landing volume holds many order elements nested under a single root element named orders. Every order element must become one row, and the cluster runs a current Databricks Runtime release.

Which read produces one row per order?

- A. `spark.read.format("xml").option("rowTag", "orders").load(path)`
- B. `spark.read.format("text").option("lineSep", "</order>").load(path)`
- C. `spark.read.format("xml").option("rowTag", "order").load(path)`
- D. `spark.read.format("xml").option("rootTag", "order").load(path)`

Correct answer: C

C. `spark.read.format("xml").option("rowTag", "order").load(path)`

Explanation

Correct answer:

`spark.read.format("xml").option("rowTag", "order").load(path)`

Why this is correct:

The XML reader's `rowTag` identifies the repeated element that should become one DataFrame row. Setting it to `order` parses each nested order element independently, even though the file has a single outer orders root. The root is a container, not the repeating record.

Why the other options are wrong:

`spark.read.format("xml").option("rowTag", "orders").load(path)`: Using the plural root tag treats the outer container as the row element, so it does not express one record per nested order. Choose the repeated element instead.

`spark.read.format("text").option("lineSep", "</order>").load(path)`: The text source reads file content as text records and does not parse the XML structure into order fields. Splitting on a closing tag is brittle and ignores nested XML semantics.

`spark.read.format("xml").option("rootTag", "order").load(path)`: The XML reader selects record elements through `rowTag`; changing a root-related setting does not identify the repeated order records. Set `rowTag` to the singular element.

References:

1. Read and write XML files | Databricks on AWS:

<https://docs.databricks.com/aws/en/query/formats/xml>

Question 2

Analysts should find tables in Catalog Explorer and search, read their descriptions and tags, and request access, but must not read any data or hold USE SCHEMA on the catalog's schemas.

Which privilege fits?

- A. Workspace access alone with Catalog Explorer enabled, which lists each catalog's tables and descriptions to every workspace user
- B. BROWSE on the catalog, which exposes its objects' metadata for discovery and access requests without granting data access
- C. SELECT on the catalog with a row filter that returns FALSE, which shows table metadata while every query returns no rows
- D. USE CATALOG and USE SCHEMA on the catalog's schemas, which show each table's metadata without granting access to its data

Correct answer: B

B. BROWSE on the catalog, which exposes its objects' metadata for discovery and access requests without granting data access

Explanation

Correct answer:

BROWSE on the catalog, which exposes its objects' metadata for discovery and access requests without granting data access

Why this is correct:

Unity Catalog's BROWSE privilege lets users discover objects and inspect metadata such as names, descriptions, and tags, and request access without granting access to the underlying data. It does not require USE SCHEMA for the catalog's schemas. That matches the discovery-only role while keeping query access separate.

Why the other options are wrong:

Workspace access alone with Catalog Explorer enabled, which lists each catalog's tables and descriptions to every workspace user: Catalog Explorer's interface cannot replace an explicit Unity Catalog privilege. Users need an appropriate catalog permission to discover its governed objects.

SELECT on the catalog with a row filter that returns FALSE, which shows table metadata while every query returns no rows: SELECT grants read access before a row filter is applied, and a filter returning no rows is not a metadata-only permission. This grants a data privilege the role should not have.

USE CATALOG and USE SCHEMA on the catalog's schemas, which show each table's metadata

without granting access to its data: Namespace usage privileges are not the dedicated metadata-discovery permission requested. They also would require grants on the schemas.

References:

1. Unity Catalog permissions model concepts | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/access-control/permissions-concepts>

Question 3

A notebook reads a PostgreSQL table over JDBC, and a reviewer found the database password in its source. It must not appear in code, Git history, notebook output, or any compute or job configuration.

How should the password be supplied?

- A. Put it in a bundle variable with a default value, so the deployed notebook receives the password from the bundle configuration
- B. Put it in a notebook widget with a default value, so the password is passed at run time and never appears in the notebook's code
- C. Read it with `os.environ` from a cluster environment variable that holds the password, so the notebook source no longer contains it
- D. Read it with `dbutils.secrets.get` from a secret scope at run time, so the value stays out of the source code and is redacted in output

Correct answer: D

D. Read it with `dbutils.secrets.get` from a secret scope at run time, so the value stays out of the source code and is redacted in output

Explanation

Correct answer:

Read it with `dbutils.secrets.get` from a secret scope at run time, so the value stays out of the source code and is redacted in output

Why this is correct:

Databricks secret scopes provide a protected way for a notebook or job to retrieve credentials at runtime with `dbutils.secrets.get`. The notebook source contains only the secret scope and key names, not the password itself, and secret values are redacted in output. This avoids placing the password in code or ordinary configuration.

Why the other options are wrong:

Put it in a notebook widget with a default value, so the password is passed at run time and never appears in the notebook's code: Notebook parameters are ordinary inputs and can be exposed to users or source history when they have defaults. Passing a secret as an ordinary parameter also risks displaying it in run metadata.

Put it in a bundle variable with a default value, so the deployed notebook receives the password from the bundle configuration: A bundle variable is deployment configuration, not a secret store; its value can be persisted in local or workspace configuration. Keep secret material in a secrets scope and retrieve it at runtime.

Read it with `os.environ` from a cluster environment variable that holds the password, so the notebook source no longer contains it: Environment variables in cluster configuration are still configuration values and can be exposed through configuration access. Reading them from Python does not make the stored password a managed secret.

References:

1. Tutorial: Create and use a Databricks secret | Databricks on AWS:

<https://docs.databricks.com/aws/en/security/secrets/example-secret-workflow>

Question 4

A notebook pages through a REST API and receives HTTP 429 with Retry-After: 60 on page 14. Authentication is valid, retries are allowed, every page must be collected, and the extract must not keep retrying forever.

How should the client respond?

- A. Retry page 14 after five seconds with a bounded retry count, and report the extract as failed if those shorter attempts run out
- B. Wait at least 60 seconds, then keep retrying page 14 until a request succeeds, and report the extract as complete once it does
- C. Wait at least 60 seconds, then retry page 14 with a bounded retry count, and report the extract as failed if those attempts run out
- D. Skip page 14 at once and continue with page 15, then report the extract as complete with page 14 logged for a later run to collect

Correct answer: C

C. Wait at least 60 seconds, then retry page 14 with a bounded retry count, and report the extract as failed if those attempts run out

Explanation

Correct answer:

Wait at least 60 seconds, then retry page 14 with a bounded retry count, and report the extract as failed if those attempts run out

Why this is correct:

Retry-After: 60 tells the client how long to wait before retrying the rate-limited request, so page 14 must not be retried sooner. A bounded retry policy respects that server guidance while ensuring the extraction eventually reports failure if the page cannot be fetched. Skipping the page would silently violate the requirement to collect every page.

Why the other options are wrong:

Wait at least 60 seconds, then keep retrying page 14 until a request succeeds, and report the extract as complete once it does: Unbounded retries can leave the extract running indefinitely, even though the requirement explicitly rules that out. The server delay should be honored within a finite retry budget.

Skip page 14 at once and continue with page 15, then report the extract as complete with page 14 logged for a later run to collect: Moving on would omit page 14 and produce an incomplete extract. A later repair is not equivalent to collecting every page in this run.

Retry page 14 after five seconds with a bounded retry count, and report the extract as failed if those shorter attempts run out: Five seconds contradicts the server's instruction to wait 60 seconds. Bounded retries are sensible only if each retry also respects the indicated backoff.

References:

1. RFC 9110 - HTTP Semantics: <https://datatracker.ietf.org/doc/rfc9110/>

Question 5

A daily job's load task has doubled in duration over three weeks. Run history shows its rows read also doubled, while its time per row and cluster size stayed flat.

What does this comparison to the baseline suggest?

- A. The slowdown points to data skew in the task, as a doubled row count tends to concentrate the new rows in a few of its partitions
- B. The slowdown needs a fresh baseline run first, as run history over three weeks does not separate volume effects from code changes
- C. The slowdown points to a code regression in the task, as input growth alone would not double a duration within three weeks
- D. The slowdown tracks input growth, which means capacity or incremental processing matters more here than a code regression

Correct answer: D

D. The slowdown tracks input growth, which means capacity or incremental processing matters more here than a code regression

Explanation

Correct answer:

The slowdown tracks input growth, which means capacity or incremental processing matters more here than a code regression

Why this is correct:

The task takes twice as long while reading twice as many rows, with time per row and cluster size unchanged. That pattern is consistent with workload growth rather than a slower implementation per unit of data. The evidence points toward capacity planning or incremental processing to control volume, not an assumed code regression.

Why the other options are wrong:

The slowdown points to a code regression in the task, as input growth alone would not double a duration within three weeks: When per-row time and compute remain stable, a proportional duration increase is exactly what a larger input can explain. The facts do not isolate a code regression.

The slowdown needs a fresh baseline run first, as run history over three weeks does not separate volume effects from code changes: Run history already supplies a useful baseline comparison: row volume doubled while processing rate and compute stayed flat. Another run may help, but a fresh baseline is not needed to recognize the observed relationship.

The slowdown points to data skew in the task, as a doubled row count tends to concentrate the

new rows in a few of its partitions: A higher row count alone does not mean those rows became concentrated into a few partitions. The stable per-row time provides no specific evidence of skew.

References:

1. Diagnose Lakeflow Jobs performance | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/diagnose-job-performance>

Question 6

A bundle defines a pipeline under `resources.pipelines` with the key `orders_etl`, plus a job that runs it. The job's pipeline task must point to the pipeline deployed in each target.

```
tasks:
- task_key: refresh
  pipeline_task:
    pipeline_id: _____
```

Which value completes the task?

- A. `${resources.pipelines.orders_etl.name}`, which resolves to the pipeline's name, and the pipeline task accepts a name in place of its ID
- B. `${resources.pipelines.orders_etl.id}`, which resolves to the ID of the pipeline that the bundle deployed to the current target
- C. `orders_etl`, which the bundle matches to the pipeline's resource key and replaces with the deployed pipeline's ID in the current target
- D. `${var.orders_etl}`, which resolves to the ID of the pipeline that the bundle deployed under that same resource key to the current target

Correct answer: B

B. `${resources.pipelines.orders_etl.id}`, which resolves to the ID of the pipeline that the bundle deployed to the current target

Explanation

Correct answer:

`${resources.pipelines.orders_etl.id}`, which resolves to the ID of the pipeline that the bundle deployed to the current target

Why this is correct:

Bundles support substitutions that let one resource refer to fields of another named resource in the same deployment. Here the pipeline task needs the deployed pipeline's ID, so `${resources.pipelines.orders_etl.id}` resolves the identifier associated with that resource key for the current target rather than embedding a workspace-specific value.

Why the other options are wrong:

orders_etl, which the bundle matches to the pipeline's resource key and replaces with the deployed pipeline's ID in the current target: The resource key is a name within the bundle configuration, not itself the deployed pipeline ID. A pipeline task expects an identifier, so the key must be resolved to its id field.

`${resources.pipelines.orders_etl.name}`, which resolves to the pipeline's name, and the pipeline task accepts a name in place of its ID: The `.name` substitution yields a display name, while the pipeline task's `pipeline_id` field requires the resource identifier. Similar substitution syntax does not make those values interchangeable.

`${var.orders_etl}`, which resolves to the ID of the pipeline that the bundle deployed under that same resource key to the current target: `${var.orders_etl}` is a custom variable and needs its own supplied value or default; it does not automatically look up the resource with that key. Resource substitutions use the `resources.pipelines` namespace.

References:

1. Substitutions and variables in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 7

A job designer makes extract depend on publish, while publish already depends on transform and transform depends on extract. The designer expects Lakeflow Jobs to work out a valid running order on its own.

What happens when the job is saved, and what must change?

- A. Saving succeeds, and at run time Lakeflow Jobs ignores the most recently added dependency so that the loop is broken before extract
- B. Saving succeeds, and each run starts the three tasks in parallel, then reruns any task whose upstream task finished after it started
- C. Saving is rejected, as the three dependencies form a cycle, so one of them must be removed to leave an acyclic graph of the job's tasks
- D. Saving succeeds, but each run stops at the first task with Upstream failed, so extract needs Run if set to All done to break the loop

Correct answer: C

C. Saving is rejected, as the three dependencies form a cycle, so one of them must be removed to leave an acyclic graph of the job's tasks

Explanation

Correct answer:

Saving is rejected, as the three dependencies form a cycle, so one of them must be removed to leave an acyclic graph of the job's tasks

Why this is correct:

Lakeflow Jobs represents dependencies as a directed acyclic graph, which must have a topological order for the scheduler to determine what can run next. Here extract waits for publish, publish waits for transform, and transform waits for extract, so no task can be first. The dependency loop must be removed before the job can be saved.

Why the other options are wrong:

Saving succeeds, but each run stops at the first task with Upstream failed, so extract needs Run if set to All done to break the loop: Run if controls how a task responds to completed upstream tasks; it cannot make a circular dependency schedulable. The cycle prevents a valid DAG before any task-level condition is evaluated.

Saving succeeds, and each run starts the three tasks in parallel, then reruns any task whose upstream task finished after it started: Parallel execution is possible only for tasks whose dependencies allow them to be ready together. A cycle does not become valid by launching its members concurrently.

Saving succeeds, and at run time Lakeflow Jobs ignores the most recently added dependency so that the loop is broken before extract: Jobs does not silently discard a dependency to repair the design. Doing so would change the specified graph, and the circular graph must be corrected explicitly.

References:

1. Lakeflow Jobs | Databricks on AWS: <https://docs.databricks.com/aws/en/jobs>

Question 8

A monorepo holds many large projects, but one team works only in the pipelines/orders directory. Cloning the whole repository into a Git folder takes too long and exceeds the workspace's limits.

How should the team create its Git folder?

- A. Create the Git folder from the whole repository, then delete the other directories and commit, so that later pulls skip them
- B. Create the Git folder from the whole repository, then enable sparse checkout on it, so that later pulls fetch that directory alone
- C. Enable sparse checkout while creating the Git folder, with a cone pattern for pipelines/orders, so only that directory is cloned
- D. Create the Git folder from a branch named pipelines/orders, which contains only that directory's files once it is cloned

Correct answer: C

C. Enable sparse checkout while creating the Git folder, with a cone pattern for pipelines/orders, so only that directory is cloned

Explanation

Correct answer:

Enable sparse checkout while creating the Git folder, with a cone pattern for pipelines/orders, so only that directory is cloned

Why this is correct:

Sparse checkout limits which repository paths are materialized in a Git folder. Configuring it during creation with cone mode for pipelines/orders avoids cloning the rest of the large monorepo into the workspace. The team keeps the Git source while reducing the checkout footprint to its working directory.

Why the other options are wrong:

Create the Git folder from a branch named pipelines/orders, which contains only that directory's files once it is cloned: A Git branch is a version of repository history, not a directory filter. Naming a branch after a path does not make the clone contain only that directory.

Create the Git folder from the whole repository, then enable sparse checkout on it, so that later pulls fetch that directory alone: Enabling sparse checkout after the full clone does not avoid the initial cost and workspace-size problem. The selection needs to be configured as the Git folder is created.

Create the Git folder from the whole repository, then delete the other directories and commit, so that later pulls skip them: Deleting other directories edits the repository working tree and does not

configure Git to omit them on future pulls. Sparse checkout controls the paths actually materialized.

References:

1. Create and manage Git folders | Databricks on AWS:

<https://docs.databricks.com/aws/en/repos/git-operations-with-repos>

Question 9

A landing volume holds hundreds of daily CSV files. A backfill names exactly two files, `orders_2026_09_01.csv` and `orders_2026_09_02.csv`, that COPY INTO has never loaded, and no other file in the folder may be loaded by this statement.

Which clause selects the source files?

- A. `PATTERN = 'orders_2026_09_01.csv'` combined with `FILES` for the second name, which merges both selection methods in one statement
- B. `FILES = ('orders_2026_09_01.csv', 'orders_2026_09_02.csv')`, which loads only those listed file names from the source path
- C. `FORMAT_OPTIONS ('files' = 'orders_2026_09_01.csv, orders_2026_09_02.csv')`, which limits the parser to reading those two names
- D. `PATTERN = 'orders_2026_09_0[1-9].csv'`, which loads the two requested files and skips the rest because they are already recorded

Correct answer: B

B. `FILES = ('orders_2026_09_01.csv', 'orders_2026_09_02.csv')`, which loads only those listed file names from the source path

Explanation

Correct answer:

`FILES = ('orders_2026_09_01.csv', 'orders_2026_09_02.csv')`, which loads only those listed file names from the source path

Why this is correct:

The `FILES` clause explicitly lists source file names for `COPY INTO`, making it suitable for a backfill that must select exactly two never-loaded files. Because the statement names only those two files, other objects in the folder are excluded regardless of whether they have been ingested previously.

Why the other options are wrong:

FORMAT_OPTIONS ('files' = 'orders_2026_09_01.csv, orders_2026_09_02.csv'), which limits the parser to reading those two names: `FORMAT_OPTIONS` configures parsing behavior, not the set of files selected for ingestion. A comma-separated string there would not express the explicit source-file list.

PATTERN = 'orders_2026_09_0[1-9].csv', which loads the two requested files and skips the rest because they are already recorded: `PATTERN` matches every filename satisfying its regular expression; it is not limited to the named two files. Reliance on prior-load tracking also fails the requirement that this statement itself select exactly those names.

PATTERN = 'orders_2026_09_01.csv' combined with FILES for the second name, which merges

both selection methods in one statement: FILES and PATTERN are alternative selection mechanisms rather than composable selectors in one COPY INTO clause. List both requested files directly.

References:

1. COPY INTO | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 10

A data quality check must isolate rows whose status is SQL NULL, but WHERE status = NULL returns no rows although many values are missing. Rows holding an empty string or the text 'NULL' must not match.

Which predicate fits?

- A. WHERE nvl(status, '') = '', because nvl turns each SQL NULL into an empty string first, so the equality test can then match it
- B. WHERE status IS NULL, because comparing any value with NULL by using = returns NULL rather than TRUE, so no row ever passes that filter
- C. WHERE coalesce(status, 'NULL') = 'NULL', because coalesce turns each SQL NULL into that text first, so the equality can match it
- D. WHERE status <=> 'NULL', because null-safe equality treats a missing value and the literal text NULL as equal to one another here

Correct answer: B

B. WHERE status IS NULL, because comparing any value with NULL by using = returns NULL rather than TRUE, so no row ever passes that filter

Explanation

Correct answer:

WHERE status IS NULL, because comparing any value with NULL by using = returns NULL rather than TRUE, so no row ever passes that filter

Why this is correct:

SQL NULL represents an unknown or absent value, so ordinary equality comparisons involving NULL evaluate to NULL rather than TRUE. A WHERE clause keeps only rows for which its predicate is TRUE. IS NULL is the predicate that explicitly tests for SQL NULL and does not confuse it with an empty string or the characters NULL.

Why the other options are wrong:

WHERE coalesce(status, 'NULL') = 'NULL', because coalesce turns each SQL NULL into that text first, so the equality can match it: After substitution, this compares the text NULL to a string, so genuine SQL NULL values may match but literal text NULL also matches. That violates the requirement to isolate only SQL NULLs.

WHERE status <=> 'NULL', because null-safe equality treats a missing value and the literal text NULL as equal to one another here: The right-hand value here is the non-null string 'NULL'; matching it finds text entries rather than SQL NULL values. Null-safe comparison does not equate those two different values.

WHERE nvl(status, '') = '', because nvl turns each SQL NULL into an empty string first, so the equality test can then match it: Because genuine empty strings also satisfy the transformed equality, this predicate combines two different data states. The check must preserve the distinction between blank text and SQL NULL.

References:

1. NULL semantics - Azure Databricks - Databricks SQL:

<https://learn.microsoft.com/en-us/azure/databricks/sql/language-manual/sql-ref-null-semantics>

Question 11

A recovery task depends on load_a and load_b and uses Run if: All failed. In today's run, load_a fails while load_b succeeds, and neither upstream task is excluded or disabled.

What happens to the recovery task?

- A. It runs, as a single failed dependency satisfies All failed whenever the other dependency is neither excluded nor disabled in the run
- B. It runs, as All failed only needs one of the listed dependencies to have failed at some point during the run
- C. It does not run and shows Excluded, because All failed requires every one of its dependencies to fail, and load_b succeeded here
- D. It shows Upstream failed and ends the run, because a failed dependency blocks every downstream task whatever its Run if value is set to

Correct answer: C

C. It does not run and shows Excluded, because All failed requires every one of its dependencies to fail, and load_b succeeded here

Explanation

Correct answer:

It does not run and shows Excluded, because All failed requires every one of its dependencies to fail, and load_b succeeded here

Why this is correct:

The All failed dependency condition is satisfied only when every listed upstream task fails. Here one task failed but the other succeeded, so the condition is false and the recovery task is marked Excluded. A single failure is enough for an At least one failed condition, which is a different setting.

Why the other options are wrong:

It shows Upstream failed and ends the run, because a failed dependency blocks every downstream task whatever its Run if value is set to: Run-if conditions exist to determine whether a downstream task is eligible after upstream results; a failed upstream does not block every task unconditionally. The configured condition is evaluated here.

It runs, as All failed only needs one of the listed dependencies to have failed at some point during the run: This describes the At least one failed behavior, not All failed. The latter requires every dependency to have failed.

It runs, as a single failed dependency satisfies All failed whenever the other dependency is neither excluded nor disabled in the run: An upstream success does not become neutral for All failed;

it makes the all-dependencies condition false. A task can be excluded even though it was not disabled or excluded upstream.

References:

1. Configure task dependencies | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/run-if>

Question 12

A job must run at 06:15 every Monday through Friday in the Europe/London time zone. The engineer uses the schedule's advanced cron syntax, which follows the Quartz format with a seconds field first.

Which expression fits?

A. 0 15 6 * * MON-FRI

B. 0 6 15 ? * MON-FRI

C. 15 6 * * MON-FRI

D. 0 15 6 ? * MON-FRI

Correct answer: D

D. 0 15 6 ? * MON-FRI

Explanation

Correct answer:

0 15 6 ? * MON-FRI

Why this is correct:

Quartz cron uses six fields here: seconds, minutes, hours, day-of-month, month, and day-of-week. The expression sets seconds to zero, minutes to 15, and hours to 6; ? leaves day-of-month unspecified because the weekday field is selected. The schedule's separate timezone setting determines that 06:15 is interpreted in Europe/London.

Why the other options are wrong:

0 15 6 * * MON-FRI: This has only five fields and uses * for day-of-month alongside a weekday selection. It is not the required six-field Quartz expression with an unspecified day-of-month.

15 6 * * MON-FRI: The seconds field is missing, so the values shift into the wrong positions and the expression does not specify the requested minute and hour in this Quartz format.

0 6 15 ? * MON-FRI: The hour and minute values are reversed: this asks for minute 6 of hour 15, rather than 06:15.

References:

1. Job | Databricks API Reference: <https://docs.databricks.com/api/jobs/v2/job>

Question 13

A Lakeflow Connect pipeline for SQL Server has its gateway and connection ready, but incremental ingestion fails to start for several tables. Some of those tables have primary keys and some do not.

What must the source database provide?

- A. Read replicas for each source table, because the connector queries a replica instead of the primary database to capture the changes
- B. Change tracking or change data capture on the source tables, with change data capture used for the tables that have no primary key
- C. Transaction log shipping to cloud storage, which the connector then reads with Auto Loader to replay each change into the destination
- D. A timestamp cursor column on each source table, which the connector compares against its last sync to find the new and changed rows

Correct answer: B

B. Change tracking or change data capture on the source tables, with change data capture used for the tables that have no primary key

Explanation

Correct answer:

Change tracking or change data capture on the source tables, with change data capture used for the tables that have no primary key

Why this is correct:

The SQL Server connector needs source-side change tracking or change data capture so it can identify changes between incremental reads. Change tracking depends on a primary key to identify changed rows; CDC can support tables without a primary key. Because this set includes both keyed and unkeyed tables, CDC is needed for the latter.

Why the other options are wrong:

Read replicas for each source table, because the connector queries a replica instead of the primary database to capture the changes: Read replicas can offload query traffic, but they do not supply the change metadata required by the connector. Replication placement does not replace source change tracking or CDC.

A timestamp cursor column on each source table, which the connector compares against its last sync to find the new and changed rows: A timestamp watermark is a common pattern for custom incremental extraction, but it is not the source capability specified for this managed SQL Server connector. It also depends on every table having a reliable cursor column.

Transaction log shipping to cloud storage, which the connector then reads with Auto Loader to replay each change into the destination: The connector's incremental path relies on supported SQL Server change metadata, not transaction-log files. Auto Loader discovers storage files and does not replay database logs into the destination.

References:

1. Configure Microsoft SQL Server for ingestion into Databricks:

<https://docs.databricks.com/aws/en/ingestion/lakeflow-connect/sql-server-source-setup>

Question 14

A vendor uploads daily.csv to a Unity Catalog volume each morning, replacing the prior day's file under the same name. A file arrival trigger on that folder ran once and has not fired since.

Which change fits this delivery pattern?

- A. Shorten the minimum time between triggers, as overwrites are detected but held for that whole interval before a new run starts
- B. Use a schedule after the vendor's usual upload window, since overwriting a file that has the same name does not start a new run
- C. Set the wait after last change to zero, as overwrites are detected but held until no further change is seen on the monitored folder
- D. Switch to a table update trigger on the volume, since a volume reports each overwritten file as a new update to a table in it

Correct answer: B

B. Use a schedule after the vendor's usual upload window, since overwriting a file that has the same name does not start a new run

Explanation

Correct answer:

Use a schedule after the vendor's usual upload window, since overwriting a file that has the same name does not start a new run

Why this is correct:

File-arrival triggers detect new files under the monitored path; replacing an existing filename does not count as a new arrival. Since this vendor overwrites daily.csv under the same name, a scheduled run after the normal delivery window matches the file delivery semantics and checks the updated contents.

Why the other options are wrong:

Shorten the minimum time between triggers, as overwrites are detected but held for that whole interval before a new run starts: The delay between trigger evaluations does not change how an overwrite is classified. A shorter interval cannot turn an existing filename into a new file-arrival event.

Set the wait after last change to zero, as overwrites are detected but held until no further change is seen on the monitored folder: The wait-after-last-change setting debounces activity after a detected arrival. It does not make same-name replacements count as a new arrival.

Switch to a table update trigger on the volume, since a volume reports each overwritten file as a new update to a table in it: A volume is file storage, not a Unity Catalog table with table commits. A table update trigger cannot monitor individual file replacements in a volume.

References:

1. Trigger jobs when new files arrive | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/file-arrival-triggers>

Question 15

Short-lived job clusters for dozens of small tasks each spend minutes acquiring cloud instances before code runs. The team stays on classic compute and accepts paying its cloud provider for a few idle instances to cut the wait.

Which feature fits this tradeoff?

- A. An instance pool with a minimum number of idle instances that the job clusters draw from, where idle instances incur no DBU charges
- B. A cluster policy that fixes one node type for all job clusters, which lets the cloud provider keep matching instances warm for the workspace
- C. Cluster autoscaling with a higher maximum worker count, which requests extra instances ahead of time so each job cluster starts sooner
- D. Spot instances with fallback to on-demand, which the cloud provider allocates faster than on-demand ones, so each job cluster starts sooner

Correct answer: A

A. An instance pool with a minimum number of idle instances that the job clusters draw from, where idle instances incur no DBU charges

Explanation

Correct answer:

An instance pool with a minimum number of idle instances that the job clusters draw from, where idle instances incur no DBU charges

Why this is correct:

An instance pool keeps a configured number of cloud instances idle and ready for clusters to acquire, reducing the delay before short-lived classic job clusters start. Idle instances incur cloud-provider charges, but Databricks does not charge DBUs for the idle pool capacity itself. This matches the team's stated tradeoff.

Why the other options are wrong:

A cluster policy that fixes one node type for all job clusters, which lets the cloud provider keep matching instances warm for the workspace: A cluster policy constrains configuration; it does not reserve already-started cloud instances. A fixed node type alone cannot eliminate instance acquisition delay.

Cluster autoscaling with a higher maximum worker count, which requests extra instances ahead of time so each job cluster starts sooner: Worker autoscaling adjusts capacity for a running cluster and does not prewarm idle instances for startup. Raising the maximum does not guarantee faster acquisition.

Spot instances with fallback to on-demand, which the cloud provider allocates faster than on-demand ones, so each job cluster starts sooner: Spot capacity can be interrupted and is not inherently allocated faster than on-demand instances. It does not provide the reserved idle capacity the team is willing to pay for.

References:

1. Connect to pools | Databricks on AWS: <https://docs.databricks.com/aws/en/compute/pool-index>

Question 16

Analysts run a few ad hoc queries each week against an operational PostgreSQL database. They need current values, data volume is modest, and the team wants no ingestion pipeline to build or maintain.

Which approach fits best?

- A. A notebook that reads the tables over JDBC each night and overwrites Delta copies, so the analysts query the values from yesterday
- B. A managed ingestion pipeline that copies the tables every hour, so the analysts query Delta copies of the data that are at most an hour old
- C. Auto Loader on a folder of nightly PostgreSQL exports, so the analysts query Delta tables that are built incrementally from each export
- D. Lakehouse Federation, with a Unity Catalog connection and a foreign catalog, so the queries read the PostgreSQL tables in place when run

Correct answer: D

D. Lakehouse Federation, with a Unity Catalog connection and a foreign catalog, so the queries read the PostgreSQL tables in place when run

Explanation

Correct answer:

Lakehouse Federation, with a Unity Catalog connection and a foreign catalog, so the queries read the PostgreSQL tables in place when run

Why this is correct:

Lakehouse Federation creates a Unity Catalog connection and foreign catalog so supported PostgreSQL tables can be queried in place. It suits occasional queries that need current values when the team does not want to operate an ingestion pipeline or maintain scheduled copies. Query federation trades local persisted copies for remote query execution.

Why the other options are wrong:

A managed ingestion pipeline that copies the tables every hour, so the analysts query Delta copies of the data that are at most an hour old: An hourly ingestion pipeline adds a managed copy and recurring pipeline operations. Its data can also be up to an hour stale, while the requirement favors current values and no ingestion process.

A notebook that reads the tables over JDBC each night and overwrites Delta copies, so the analysts query the values from yesterday: A nightly JDBC copy introduces custom maintenance and serves yesterday's snapshot. This misses the need for current data and the no-pipeline operating model.

Auto Loader on a folder of nightly PostgreSQL exports, so the analysts query Delta tables that are built incrementally from each export: Auto Loader ingests files from object storage; it does not connect directly to a live PostgreSQL database. Nightly exports would still be copies rather than current reads.

References:

1. What is query federation? | Databricks on AWS:

<https://docs.databricks.com/aws/en/query-federation/database-federation>

Question 17

A CI pipeline deploys a bundle whose job uses the variable `warehouse_id`. The value differs per CI environment and must be supplied at deploy time without editing `databricks.yml` or committing any new file.

How should CI supply the value?

- A. Set `warehouse_id` in a `vars.yml` file at the project root that the CI step generates, which the bundle then reads during each deployment
- B. Set a job parameter named `warehouse_id` in the CI step, which the bundle then substitutes for `${var.warehouse_id}` during deployment
- C. Set the environment variable `DATABRICKS_WAREHOUSE_ID` in the CI step, which the CLI then maps to the bundle variable named `warehouse_id`
- D. Set the environment variable `BUNDLE_VAR_warehouse_id` in the CI step before that step runs `databricks bundle deploy` for the target

Correct answer: D

D. Set the environment variable `BUNDLE_VAR_warehouse_id` in the CI step before that step runs `databricks bundle deploy` for the target

Explanation

Correct answer:

Set the environment variable `BUNDLE_VAR_warehouse_id` in the CI step before that step runs `databricks bundle deploy` for the target

Why this is correct:

Declarative Automation Bundles map environment variables prefixed with `BUNDLE_VAR_` to custom bundle variables. Setting `BUNDLE_VAR_warehouse_id` before the CLI deploy command supplies the deployment-time value for `${var.warehouse_id}` without editing the YAML or adding a committed override file.

Why the other options are wrong:

Set the environment variable `DATABRICKS_WAREHOUSE_ID` in the CI step, which the CLI then maps to the bundle variable named `warehouse_id`: Bundle custom variables are mapped from environment variables with the `BUNDLE_VAR_` prefix. The warehouse-specific environment variable does not populate the unrelated bundle variable name.

Set `warehouse_id` in a `vars.yml` file at the project root that the CI step generates, which the bundle then reads during each deployment: A generated variables file would be another file-based override, not the requested environment-variable mechanism. It also adds deployment setup that the stem explicitly avoids.

Set a job parameter named `warehouse_id` in the CI step, which the bundle then substitutes for `${var.warehouse_id}` during deployment: Job parameters are supplied when a deployed job runs; bundle variables are resolved when the bundle is deployed. A run parameter does not substitute for `${var.warehouse_id}` in configuration.

References:

1. Substitutions and variables in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 18

A Lakeflow Spark Declarative Pipelines table uses `expect_or_drop`, and silver row counts look low. The engineer needs the number of records each expectation failed across recent updates, queried with SQL.

Where are these counts recorded?

- A. In the `system.access.audit` table, which records each dropped record together with the expectation that removed it from the silver table
- B. In the pipeline event log, whose `flow_progress` events carry data quality details with passed and failed records for each expectation
- C. In the pipeline's checkpoint directory, whose offset files list the records that each expectation dropped during each pipeline update
- D. In the silver table's `DESCRIBE HISTORY` output, whose operation metrics list the records that each expectation removed during the update

Correct answer: B

B. In the pipeline event log, whose `flow_progress` events carry data quality details with passed and failed records for each expectation

Explanation

Correct answer:

In the pipeline event log, whose `flow_progress` events carry data quality details with passed and failed records for each expectation

Why this is correct:

Lakeflow Spark Declarative Pipelines emits data-quality information in its event log. `flow_progress` events include `data_quality` details such as the expectation name and passed or failed record counts, which can be queried with SQL for recent updates. The log supplies metrics rather than the actual dropped row payloads.

Why the other options are wrong:

In the `system.access.audit` table, which records each dropped record together with the expectation that removed it from the silver table: Governance and access events are not records of rows rejected by a pipeline expectation. The audit system table does not serve as the pipeline's expectation-metrics store.

In the pipeline's checkpoint directory, whose offset files list the records that each expectation dropped during each pipeline update: Streaming checkpoints track processing progress and recovery state, not queryable per-expectation row counts. The event log is the supported monitoring record.

In the silver table's DESCRIBE HISTORY output, whose operation metrics list the records that each expectation removed during the update: Delta history summarizes table operations and some operation metrics, but expectation-level counts are attached to pipeline event-log updates. Use the pipeline's structured event records for that detail.

References:

1. Pipeline event log schema | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/ldp/monitor-event-log-schema>

Question 19

A governed tag named sensitivity has the allowed values low, medium, and high. A steward holding ASSIGN on the tag and APPLY TAG on a table tries to set sensitivity = restricted on one of its columns.

What happens?

- A. The assignment succeeds as an ordinary tag with the same key, because an unapproved value is kept apart from the governed tag's values
- B. The assignment succeeds and adds restricted to the allowed values, as holding ASSIGN lets a steward extend the tag's value list
- C. The assignment succeeds, but ABAC policies ignore that column, as they match only the values on the governed tag's allowed list
- D. The assignment is rejected, because a governed tag accepts only its defined values, so a tag manager must add the new value first

Correct answer: D

D. The assignment is rejected, because a governed tag accepts only its defined values, so a tag manager must add the new value first

Explanation

Correct answer:

The assignment is rejected, because a governed tag accepts only its defined values, so a tag manager must add the new value first

Why this is correct:

Governed tags restrict assignments to values declared in the tag's allowed-value list. Since restricted is not among low, medium, and high, the attempted assignment is rejected until a tag manager adds that value. ASSIGN and APPLY TAG authorize applying an allowed value, but do not expand the governance definition.

Why the other options are wrong:

The assignment succeeds and adds restricted to the allowed values, as holding ASSIGN lets a steward extend the tag's value list: ASSIGN authorizes tagging with defined values; permission to use a tag does not imply permission to edit its allowed-value set. A manager must update the governed tag definition.

The assignment succeeds as an ordinary tag with the same key, because an unapproved value is kept apart from the governed tag's values: An invalid assignment does not silently create an ordinary ungoverned tag with the same key. Governed-value enforcement rejects it.

The assignment succeeds, but ABAC policies ignore that column, as they match only the values on the governed tag's allowed list: Policy evaluation uses governed tag values only after a permitted assignment. The invalid value must first be added to the allowed list.

References:

1. Create and manage governed tags | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/admin/governed-tags/manage-governed-tags>

Question 20

A stage shows task duration median 18 s and max 21 s, input median 110 MB and max 125 MB, and GC time median 80 ms and max 100 ms. No task failed or spilled.

What does this evidence support?

- A. Too many small tasks in this stage, as inputs near 110 MB are below the 128 MB split target, so partitions should be coalesced
- B. No meaningful skew, spill, or garbage collection pressure appears in this stage, so the job's bottleneck most likely lies elsewhere
- C. Moderate skew in this stage, as the slowest task ran about 17 percent longer than the median, which warrants AQE skew handling
- D. Too few shuffle partitions in this stage, because task durations that are close to each other show each task processes too much data

Correct answer: B

B. No meaningful skew, spill, or garbage collection pressure appears in this stage, so the job's bottleneck most likely lies elsewhere

Explanation

Correct answer:

No meaningful skew, spill, or garbage collection pressure appears in this stage, so the job's bottleneck most likely lies elsewhere

Why this is correct:

The largest task is only 3 seconds above the median and reads modestly more input; with no failed tasks or spill, these measurements do not show material skew or memory pressure. GC time is also small relative to task duration. The stage evidence therefore points away from these Spark-stage symptoms, so investigate other stages or external waits.

Why the other options are wrong:

Too few shuffle partitions in this stage, because task durations that are close to each other show each task processes too much data: Too few partitions can create oversized tasks, but the observed tasks are similar in duration and input and show no spill. This evidence does not establish partition count as the problem.

Too many small tasks in this stage, as inputs near 110 MB are below the 128 MB split target, so partitions should be coalesced: An input split size rule of thumb does not make a 110 MB task inherently too small or prove that task overhead dominates. The measurements show no evidence that coalescing would help.

Moderate skew in this stage, as the slowest task ran about 17 percent longer than the median, which warrants AQE skew handling: A max 21-second task against an 18-second median is a small spread, and the input spread is similarly modest. The evidence does not indicate the pronounced task imbalance that skew handling targets.

References:

1. Skew and spill | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/long-spark-stage-page>

Question 21

Executors fail with out-of-memory errors while processing a few very large shuffle partitions of about 2 GB each. The driver is healthy, no collect or toPandas call runs, and every input row must be kept.

Which TWO changes directly address the failure?

- A.** Increase the driver's memory, so that the driver can hold the shuffle data from the large partitions on behalf of the executors
- B.** Add executor cores while keeping executor memory the same, so that each large partition finishes before the memory runs out
- C.** Give each concurrent task more executor memory, for example by choosing a worker type that has more memory for each core
- D.** Make each shuffle task process less data, for example by raising the shuffle partition count or enabling AQE skew handling
- E.** Filter out the rows in the largest partitions, so that each shuffle task stays within the executor memory it already has

Correct answers: C, D

- C.** Give each concurrent task more executor memory, for example by choosing a worker type that has more memory for each core
- D.** Make each shuffle task process less data, for example by raising the shuffle partition count or enabling AQE skew handling

Explanation

Correct answer:

Give each concurrent task more executor memory, for example by choosing a worker type that has more memory for each core

Make each shuffle task process less data, for example by raising the shuffle partition count or enabling AQE skew handling

Why this is correct:

The failure is in executor tasks processing oversized shuffle partitions, so address memory per concurrent task and amount of data per task. A worker type with more memory per core gives each task a larger memory share; more shuffle partitions or AQE skew handling can split large work into smaller tasks while preserving all input rows. Driver memory does not solve executor-side pressure.

Why the other options are wrong:

Add executor cores while keeping executor memory the same, so that each large partition

finishes before the memory runs out: Adding executor cores at the same memory increases task concurrency and can leave less memory available per task. It does not make a single 2 GB shuffle partition smaller.

Filter out the rows in the largest partitions, so that each shuffle task stays within the executor memory it already has: Filtering rows would discard required input and change the result, rather than fixing the resource pressure. The stem explicitly requires preserving every row.

Increase the driver's memory, so that the driver can hold the shuffle data from the large partitions on behalf of the executors: The driver is healthy and shuffle tasks fail on executors, so increasing driver memory targets the wrong process. Executors hold the data being processed by those tasks.

References:

1. Spark memory issues | Databricks on AWS:

<https://docs.databricks.com/aws/en/optimizations/spark-ui-guide/spark-memory-issues>

Question 22

A Lakeflow Connect pipeline must keep destination tables matching the source's current state through incremental runs, with updates overwriting rows and deletes removing them. No earlier versions are needed, and storage growth should stay minimal.

Which history setting fits these needs?

- A. Leave history tracking off and enable change data feed on the destination tables, so updates and deletes apply to current rows
- B. Leave history tracking off and run a full refresh on each update, so the destination is rebuilt to match the source's current state
- C. Leave history tracking off, which applies SCD type 1 behavior, so updates and deletes are applied directly to the current rows
- D. Turn history tracking on as SCD type 2, then filter every query on `__END_AT IS NULL` so it reads the current version of each row

Correct answer: C

C. Leave history tracking off, which applies SCD type 1 behavior, so updates and deletes are applied directly to the current rows

Explanation

Correct answer:

Leave history tracking off, which applies SCD type 1 behavior, so updates and deletes are applied directly to the current rows

Why this is correct:

With history tracking disabled, the connector applies SCD type 1 semantics: each update changes the current destination row, and deletes are reflected in the current state. Since the requirement explicitly rejects historical versions and wants lower storage growth, retaining only the latest state fits the intended model.

Why the other options are wrong:

Leave history tracking off and enable change data feed on the destination tables, so updates and deletes apply to current rows: Change Data Feed exposes changes for downstream consumers but does not itself define this connector's destination history mode. SCD type 1 behavior comes from leaving history tracking off.

Leave history tracking off and run a full refresh on each update, so the destination is rebuilt to match the source's current state: Full refresh rebuilds the destination and is not necessary for normal incremental updates when history tracking is off. Rebuilding each time would add cost rather than provide the efficient current-state synchronization requested.

Turn history tracking on as SCD type 2, then filter every query on `__END_AT IS NULL` so it reads the current version of each row: SCD type 2 preserves historical versions and increases storage, which conflicts with the explicit no-history requirement. Filtering to current rows at query time does not remove the retained history.

References:

1. Enable history tracking (SCD type 2) | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/ingestion/lakeflow-connect/scd>

Question 23

A company's Airflow deployment already decides when every data job starts, based on dependencies outside Databricks. One Databricks job must start only when Airflow requests it and never on its own schedule.

How should the job be configured?

- A. With a paused scheduled trigger, which Airflow unpauses for a single run and then pauses again once that run has finished
- B. With no trigger, and Airflow starts each run through the Jobs API submit call, passing a copy of the job's task definitions
- C. With no trigger, and Airflow starts each run through the Jobs API run-now call, for example with its Databricks run-now operator
- D. With a continuous trigger, which keeps the job ready so that each Airflow request is served by the run that is already active

Correct answer: C

C. With no trigger, and Airflow starts each run through the Jobs API run-now call, for example with its Databricks run-now operator

Explanation

Correct answer:

With no trigger, and Airflow starts each run through the Jobs API run-now call, for example with its Databricks run-now operator

Why this is correct:

A job without a schedule or trigger is started on demand, including through the Jobs API run-now operation. That API starts a run of the job as configured, so Airflow can remain the sole scheduler and invoke the Databricks job after its external dependency conditions are met.

Why the other options are wrong:

With no trigger, and Airflow starts each run through the Jobs API submit call, passing a copy of the job's task definitions: submit starts a one-time run by submitting task definitions; it is not the endpoint for running the already configured job. run-now invokes the existing job definition.

With a paused scheduled trigger, which Airflow unpauses for a single run and then pauses again once that run has finished: A paused schedule is unnecessary and adds state changes around every run. Leaving the job unscheduled and invoking it directly preserves the requested external orchestration.

With a continuous trigger, which keeps the job ready so that each Airflow request is served by the run that is already active: Continuous scheduling keeps a job running or restarts it according to its

continuous-job semantics. It is not an inert on-demand endpoint and would add autonomous execution behavior.

References:

1. Automate jobs with schedules and triggers | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/triggers>

2. Jobs API 2.0 | Databricks on AWS: <https://docs.databricks.com/aws/en/reference/jobs-2.0-api>

Question 24

A task with `max_retries` set to 3 and a five-minute minimum retry interval fails its first attempt. An operator sees the run still active with the task showing **Waiting for retry** and considers canceling the run.

What does this status mean?

- A. The attempt failed, and the whole job run will restart from its first task once the retry interval passes, rerunning completed tasks
- B. The attempt failed, and the task will retry only after an operator approves the retry from the run page within the retry interval
- C. The attempt timed out, and the task waits for the job's queue to free a slot before a new run of the whole job can start
- D. The attempt failed, and the task will start another attempt once the retry interval passes, so the run is continuing as configured

Correct answer: D

D. The attempt failed, and the task will start another attempt once the retry interval passes, so the run is continuing as configured

Explanation

Correct answer:

The attempt failed, and the task will start another attempt once the retry interval passes, so the run is continuing as configured

Why this is correct:

Waiting for retry means the failed task has another configured attempt pending and is waiting for its retry interval. The run remains active while Lakeflow Jobs waits, then starts the task again within its retry budget. It does not restart completed upstream work or require an operator to approve the next attempt.

Why the other options are wrong:

The attempt failed, and the whole job run will restart from its first task once the retry interval passes, rerunning completed tasks: Task retries retry that task according to its policy; they do not automatically restart the entire job from the first task. Completed upstream tasks remain completed unless the operator repairs or reruns them.

The attempt failed, and the task will retry only after an operator approves the retry from the run page within the retry interval: The configured retry policy is automatic, so an operator approval is not required for the next attempt. The interval defines timing, not an approval window.

The attempt timed out, and the task waits for the job's queue to free a slot before a new run of the

whole job can start: The task has already failed and is waiting for its next configured attempt within the same run; it is not a timeout or a queue wait for a fresh job run.

References:

1. Configure and edit tasks in Lakeflow Jobs | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/configure-task>

Question 25

A team wants to adopt liquid clustering on a Delta table that is partitioned by event_date and maintained with a nightly OPTIMIZE ZORDER BY (customer_id). Queries filter on both of these columns.

Which plan fits?

- A. Replace the partitioning with CLUSTER BY (event_date, customer_id) and drop the ZORDER step, since clustering takes over from them
- B. Keep the event_date partitioning and switch the ZORDER step to CLUSTER BY AUTO, so keys are picked within each partition
- C. Keep the event_date partitioning and add CLUSTER BY (customer_id), so clustering organizes the data files inside each existing partition
- D. Keep the nightly OPTIMIZE ZORDER BY (customer_id) and add CLUSTER BY (event_date), so each of the two techniques covers one column

Correct answer: A

A. Replace the partitioning with CLUSTER BY (event_date, customer_id) and drop the ZORDER step, since clustering takes over from them

Explanation

Correct answer:

Replace the partitioning with CLUSTER BY (event_date, customer_id) and drop the ZORDER step, since clustering takes over from them

Why this is correct:

Liquid clustering is intended to replace legacy partitioning and Z-ordering as the table's data layout strategy. Since both event_date and customer_id filter the workload, clustering on both lets the layout adapt around both query dimensions. The partition and nightly ZORDER configuration should therefore be removed when adopting this approach.

Why the other options are wrong:

Keep the event_date partitioning and switch the ZORDER step to CLUSTER BY AUTO, so keys are picked within each partition: Partitioning remains a separate layout technique and is not layered under liquid clustering. CLUSTER BY AUTO does not make the existing partition scheme part of the new clustering strategy.

Keep the event_date partitioning and add CLUSTER BY (customer_id), so clustering organizes the data files inside each existing partition: Liquid clustering is not defined as file organization within each existing partition. Retaining partitioning alongside it combines incompatible layout approaches.

Keep the nightly OPTIMIZE ZORDER BY (customer_id) and add CLUSTER BY (event_date), so each of the two techniques covers one column: ZORDER and liquid clustering are alternatives for organizing Delta data, not complementary steps for splitting keys between them. Keeping both would retain the legacy layout operation the migration is meant to replace.

References:

1. Use liquid clustering for tables | Databricks on AWS:

<https://docs.databricks.com/aws/en/delta/clustering>

Question 26

A team builds a streaming table from an append-only bronze table in a Lakeflow Spark Declarative Pipeline. Months later, they change the streaming table's query to add a derived column.

Which TWO statements are true?

- A. Rows processed before the change keep their earlier values, and the rows that arrive after the change use the new query
- B. Existing rows are recomputed automatically with the new query during the next regular triggered update of the pipeline
- C. A full refresh of the streaming table reprocesses all of the available source data with the new version of the query
- D. The bronze source must switch from append-only to a change data feed before the new version of the query can take effect
- E. The streaming table converts to a materialized view on the next update, so the query change also applies to earlier rows

Correct answers: A, C

- A. Rows processed before the change keep their earlier values, and the rows that arrive after the change use the new query
- C. A full refresh of the streaming table reprocesses all of the available source data with the new version of the query

Explanation

Correct answer:

Rows processed before the change keep their earlier values, and the rows that arrive after the change use the new query

A full refresh of the streaming table reprocesses all of the available source data with the new version of the query

Why this is correct:

A streaming table's regular incremental update processes new source data under the current definition; it does not revisit already processed records just because the query changed. A full refresh reprocesses all available source data using the current definition. Thus old outputs remain as they were until a full refresh, while newly arriving rows use the added derived column logic.

Why the other options are wrong:

Existing rows are recomputed automatically with the new query during the next regular triggered

update of the pipeline: An ordinary incremental update does not automatically recompute prior streaming-table rows. Historical rows require a full refresh if they must reflect the changed query.

The bronze source must switch from append-only to a change data feed before the new version of the query can take effect: Change Data Feed is not required to apply a changed transformation to an append-only source. The relevant choice is whether to process only new data or perform a full refresh.

The streaming table converts to a materialized view on the next update, so the query change also applies to earlier rows: Streaming tables and materialized views have distinct refresh behavior; modifying a query does not transform one object type into the other. The intended full-refresh behavior must be requested explicitly.

References:

1. How do pipelines refresh? | Databricks on AWS:

<https://docs.databricks.com/aws/en/ldp/concepts/refresh>

Question 27

Two DataFrames hold customer records. `west` has `id`, `name`, `email`, and `loyalty_tier`, while `east` has `id`, `email`, and `name` in that order. The result must align columns by name and show `NULL` `loyalty_tier` for east rows.

Which expression combines them?

- A. `west.union(east)`
- B. `west.unionByName(east, allowMissingColumns=True)`
- C. `west.unionByName(east)`
- D. `west.join(east, "id", "outer")`

Correct answer: B

B. `west.unionByName(east, allowMissingColumns=True)`

Explanation

Correct answer:

`west.unionByName(east, allowMissingColumns=True)`

Why this is correct:

`unionByName` matches fields by column name instead of by position, so the differing east column order does not misplace values. With `allowMissingColumns=True`, the missing `loyalty_tier` field is filled with `NULL` for east rows. A union therefore appends records while retaining the combined schema.

Why the other options are wrong:

`west.unionByName(east)`: Unless `allowMissingColumns=True` is supplied, the name-aware union still expects compatible schemas and does not create the missing `loyalty_tier` field. Add that flag so east rows receive `NULL` for the absent field.

`west.union(east)`: Positional union pairs the second field with the second field, even though east orders its columns differently. That can assign email values to the wrong output column.

`west.join(east, "id", "outer")`: A join combines matching rows side by side; it does not append both customer datasets as separate rows. It can also multiply data for duplicate IDs.

References:

1. DataFrame.unionByName | Apache Spark: <https://spark.apache.org/docs/4.2.0-preview5/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.unionByName.html>

Question 28

A supplier starts adding unexpected fields to a JSON feed. The team must keep what was received for reprocessing and still publish a stable, cleaned schema to consumers without changing the feed.

Where should the raw records and the cleansing logic live?

- A. Land the records as received in bronze with schema evolution, and expose bronze directly to consumers once the new fields settle
- B. Land the records as received in bronze, then apply the cleansing and a stable consumer-facing schema when building the silver tables from it
- C. Land the records directly in gold with the stable schema, and rebuild any dropped fields later from the supplier's archived deliveries of the feed
- D. Land the records as received in bronze, and let each consumer apply its own cleansing when it reads bronze instead of a shared silver layer

Correct answer: B

B. Land the records as received in bronze, then apply the cleansing and a stable consumer-facing schema when building the silver tables from it

Explanation

Correct answer:

Land the records as received in bronze, then apply the cleansing and a stable consumer-facing schema when building the silver tables from it

Why this is correct:

The medallion pattern keeps ingested source records in bronze so teams can replay and reinterpret them, then applies validation, cleansing, and a stable schema in silver. This preserves unexpected supplier fields for future reprocessing while providing consumers a curated contract. Exposing evolving bronze directly would transfer schema instability to every consumer.

Why the other options are wrong:

Land the records as received in bronze, and let each consumer apply its own cleansing when it reads bronze instead of a shared silver layer: Putting cleansing responsibility on each consumer duplicates transformation logic and can produce inconsistent schemas. Bronze is the replayable source layer, not the stable consumer interface.

Land the records directly in gold with the stable schema, and rebuild any dropped fields later from the supplier's archived deliveries of the feed: Writing directly to gold discards the raw landing layer needed to reprocess unexpected fields. Depending on a supplier archive also weakens control over recovery and replay.

Land the records as received in bronze with schema evolution, and expose bronze directly to consumers once the new fields settle: Preserving new fields in bronze supports replay, but exposing that layer directly leaves consumers coupled to changing input structure. A curated silver layer provides the stable schema boundary.

References:

1. What is medallion lakehouse architecture? | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/lakehouse/medallion>

Question 29

A nightly production notebook runs on a shared all-purpose cluster that stays up all day for interactive users. Finance wants the nightly workload's compute cost lowered without changing its code, libraries, or runtime version.

Which change fits this goal?

- A. Run the notebook as a job task on serverless compute for notebooks, which keeps the current runtime version pinned for the workload
- B. Move the shared all-purpose cluster onto an instance pool, so the notebook's workers are billed at the pool rate instead of the all-purpose rate
- C. Run the notebook as a job task on the same shared all-purpose cluster, which bills the task at the jobs compute rate while it runs
- D. Run the notebook as a job task on its own jobs compute, which is billed at a lower rate than all-purpose compute and stops when the task ends

Correct answer: D

D. Run the notebook as a job task on its own jobs compute, which is billed at a lower rate than all-purpose compute and stops when the task ends

Explanation

Correct answer:

Run the notebook as a job task on its own jobs compute, which is billed at a lower rate than all-purpose compute and stops when the task ends

Why this is correct:

Dedicated jobs compute is provisioned for the task and terminates when the workload finishes, unlike the all-purpose cluster kept running for interactive users. Databricks pricing distinguishes jobs compute from all-purpose compute, so moving the nightly task to its own job compute can lower its compute rate and avoid paying to keep the shared cluster up for it. The task's code, libraries, and runtime can remain configured for the workload.

Why the other options are wrong:

Run the notebook as a job task on the same shared all-purpose cluster, which bills the task at the jobs compute rate while it runs: This configuration keeps the workload on all-purpose compute, so it retains all-purpose billing. The shared cluster also remains available and billable outside the notebook's nightly runtime.

Run the notebook as a job task on serverless compute for notebooks, which keeps the current runtime version pinned for the workload: Serverless compute may not support pinning an arbitrary existing runtime version, so it conflicts with the requirement to preserve the runtime. A job can instead

use classic dedicated jobs compute.

Move the shared all-purpose cluster onto an instance pool, so the notebook's workers are billed at the pool rate instead of the all-purpose rate: Pools reduce instance acquisition time; they do not change the compute billing category into a lower pool rate. A pool can also incur cloud provider charges for idle instances.

References:

1. Configure compute for jobs | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/compute>

Question 30

A team maintains a dbt project in Git that builds its gold models and has no dbt Cloud account. Lakeflow Jobs must run the project after an ingestion task on an existing SQL warehouse, without any wrapper code.

Which task type fits?

- A. A dbt platform task that triggers the project's job in a dbt Cloud account, so the models are built on the warehouse after ingestion
- B. A SQL task that runs the dbt-compiled SQL files committed to the repository, so the warehouse executes the models after ingestion
- C. A dbt task that points to the project in Git and uses that SQL warehouse, so dbt's generated SQL runs there once ingestion finishes
- D. A notebook task that runs dbt build through the dbt CLI on the job cluster, so the models are built on the warehouse after ingestion

Correct answer: C

C. A dbt task that points to the project in Git and uses that SQL warehouse, so dbt's generated SQL runs there once ingestion finishes

Explanation

Correct answer:

A dbt task that points to the project in Git and uses that SQL warehouse, so dbt's generated SQL runs there once ingestion finishes

Why this is correct:

Lakeflow Jobs provides a dbt task type that can reference a project in Git and execute its models using a SQL warehouse. This runs the project directly after the upstream ingestion dependency without requiring a dbt Cloud account or wrapper code. The warehouse executes the generated SQL while the task orchestrates the project.

Why the other options are wrong:

A notebook task that runs dbt build through the dbt CLI on the job cluster, so the models are built on the warehouse after ingestion: Invoking a CLI from a notebook requires custom orchestration code and does not meet the no-wrapper requirement. It may also run the CLI on a cluster rather than the specified warehouse task path.

A SQL task that runs the dbt-compiled SQL files committed to the repository, so the warehouse executes the models after ingestion: Executing only compiled SQL bypasses dbt's project execution model, including its selection and build behavior. The dedicated dbt task understands the project and its configuration.

A dbt platform task that triggers the project's job in a dbt Cloud account, so the models are built on the warehouse after ingestion: This task type integrates with dbt Cloud, which the team does not have. The standard dbt task runs the Git project within Lakeflow Jobs.

References:

1. dbt task for jobs | Databricks on AWS: <https://docs.databricks.com/aws/en/jobs/tasks/dbt>

Question 31

A task's run page shows the status **Timed out after exactly 30 minutes**, although its logs show no error, and the last output line reports steady progress through a large input.

What most likely ended the task?

- A. The task ran past its configured timeout, so Databricks stopped it even though its work was still progressing normally on the large input
- B. The task's max_retries ran out after repeated failures within 30 minutes, so the final attempt was recorded as Timed out
- C. An upstream task failed after 30 minutes, so Databricks stopped this task because its Run if condition was no longer met by the run
- D. The cluster's auto termination period elapsed, so the idle driver shut down while the task was still processing the large input

Correct answer: A

A. The task ran past its configured timeout, so Databricks stopped it even though its work was still progressing normally on the large input

Explanation

Correct answer:

The task ran past its configured timeout, so Databricks stopped it even though its work was still progressing normally on the large input

Why this is correct:

The run page explicitly reports Timed out at the 30-minute boundary while logs show work continuing without an error. Lakeflow Jobs applies a configured task timeout and stops the task when that duration is reached, regardless of whether the workload is still making progress. The likely cause is therefore the task's timeout setting.

Why the other options are wrong:

The cluster's auto termination period elapsed, so the idle driver shut down while the task was still processing the large input: Auto termination applies to idle compute, whereas the task is actively processing input and reporting progress. The exact timeout status points to the run-duration limit instead.

The task's max_retries ran out after repeated failures within 30 minutes, so the final attempt was recorded as Timed out: Exhausting retries follows unsuccessful attempts; the stem reports steady progress and a timeout at an exact duration, not repeated task failures. Retry settings do not explain this status by themselves.

An upstream task failed after 30 minutes, so Databricks stopped this task because its Run if

condition was no longer met by the run: An upstream dependency condition is evaluated around task eligibility, not used to stop a currently progressing task after 30 minutes. The run reports the specific Timed out state.

References:

1. Configure and edit tasks in Lakeflow Jobs | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/configure-task>

Question 32

A For each task iterates over a JSON array of region names. Its nested notebook task's region parameter must receive only the current element.

```
for_each_task:
  inputs: '["emea", "apac", "amer"]'
  task:
    task_key: load_region
    notebook_task:
      notebook_path: ./load_region.py
      base_parameters:
        region: _____
```

Which value completes the parameter?

- A.** `{{job.parameters.region}}`, which resolves to the region value that the For each task assigns to each of its nested task iterations
- B.** `{{tasks.load_region.values.region}}`, which resolves to the element that the nested task stores as a task value during its iteration
- C.** `{{inputs}}`, which resolves to the whole array for each iteration, so the notebook then selects its own element from the full array value
- D.** `{{input}}`, which resolves to the array element for the current iteration, so each nested run receives exactly one region name as its value

Correct answer: D

D. `{{input}}`, which resolves to the array element for the current iteration, so each nested run receives exactly one region name as its value

Explanation

Correct answer:

`{{input}}`, which resolves to the array element for the current iteration, so each nested run receives exactly one region name as its value

Why this is correct:

Inside a For each task, `{{input}}` is the dynamic value for the current iteration. With a JSON array of region names, each nested notebook run receives one string at a time. The loop therefore supplies the per-iteration value directly instead of passing the entire array for the notebook to interpret.

Why the other options are wrong:

`{{inputs}}`, which resolves to the whole array for each iteration, so the notebook then selects its own element from the full array value: Passing all inputs gives the nested task the complete array rather than the current element. The For each task already performs iteration, so downstream code

should receive its single current value.

{{tasks.load_region.values.region}}, which resolves to the element that the nested task stores as a task value during its iteration: Task values are values explicitly set and retrieved during a run, not the automatic loop variable. This reference would not select the current region from the For each inputs.

{{job.parameters.region}}, which resolves to the region value that the For each task assigns to each of its nested task iterations: Job parameters are run-level settings shared by the job, not the value for each iteration. The loop's dynamic reference supplies each element individually.

References:

1. Use a For each task to run another task in a loop | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/tasks/for-each>

Question 33

The table `shipments` has five rows. `carrier_code` is `NULL` in two of them, and the three non-null values are `UPS`, `UPS`, and `DHL`.

```
SELECT COUNT(*), COUNT(carrier_code), COUNT(DISTINCT carrier_code)
FROM shipments;
```

What does the query return?

- A. 5, 3, and 3, because the distinct count skips the `NULL`s but still counts each non-null value once for every row that holds that value
- B. 5, 3, and 2, because `COUNT(*)` counts every row, while the column counts skip `NULL`s and the distinct count also merges duplicates
- C. 5, 5, and 3, as every form of `COUNT` counts `NULL`s as values, and the distinct count also treats the two `NULL`s as one additional value
- D. 3, 3, and 2, because `COUNT(*)` skips each row that holds a `NULL` in some column, just as the column count does for `carrier_code` values

Correct answer: B

B. 5, 3, and 2, because `COUNT(*)` counts every row, while the column counts skip `NULL`s and the distinct count also merges duplicates

Explanation

Correct answer:

5, 3, and 2, because `COUNT(*)` counts every row, while the column counts skip `NULL`s and the distinct count also merges duplicates

Why this is correct:

`COUNT(*)` counts all five rows, including those with `NULL` values. `COUNT(carrier_code)` counts only the three non-`NULL` entries, and `COUNT(DISTINCT carrier_code)` counts the two unique non-`NULL` values, `UPS` and `DHL`. SQL aggregate `COUNT` excludes `NULL` arguments, including for a distinct count.

Why the other options are wrong:

5, 3, and 3, because the distinct count skips the `NULL`s but still counts each non-null value once for every row that holds that value: Distinct counting merges repeated values rather than counting each occurrence. The two `UPS` rows contribute one unique code, so the third result is two, not three.

5, 5, and 3, as every form of `COUNT` counts `NULL`s as values, and the distinct count also treats the two `NULL`s as one additional value: `COUNT(*)` counts rows, but a column expression such as `COUNT(carrier_code)` ignores `NULL`s. The distinct column count also does not add `NULL` as another

distinct value.

3, 3, and 2, because COUNT(*) skips each row that holds a NULL in some column, just as the column count does for carrier_code values: The star form counts rows regardless of whether a particular column is NULL. Only the expression-based counts here omit rows with NULL carrier_code.

References:

1. count aggregate function | Databricks on AWS:

<https://docs.databricks.com/aws/ja/sql/language-manual/functions/count>

Question 34

A daily job writes about 2 GB to a Delta table, but each run creates about 4,000 files of a few hundred kilobytes, slowing readers. The team will fix the file count in the writing job first.

Which change should be tested first, then remeasured?

- A. Call cache on the DataFrame before the write, so the write reads its data from memory and produces fewer files in each daily run
- B. Run VACUUM after each write, so the small files from each run are merged into larger files before readers query the table again
- C. Call coalesce with a small partition count on the DataFrame before the write, so each run produces far fewer and larger files in the table
- D. Raise spark.sql.shuffle.partitions for the job, so the final stage writes the 2 GB across more tasks and more evenly sized files in the table

Correct answer: C

C. Call coalesce with a small partition count on the DataFrame before the write, so each run produces far fewer and larger files in the table

Explanation

Correct answer:

Call coalesce with a small partition count on the DataFrame before the write, so each run produces far fewer and larger files in the table

Why this is correct:

The many tiny output files suggest the write is launching too many small output tasks for this workload. Coalescing the DataFrame to fewer partitions before the write is a direct first experiment to reduce writer-task and file counts; then inspect file sizes and reader performance. This is a measured tuning step, not a universal fixed partition count.

Why the other options are wrong:

Run VACUUM after each write, so the small files from each run are merged into larger files before readers query the table again: VACUUM removes eligible obsolete files; it does not combine the current small data files into larger ones. File compaction is a separate layout operation.

Call cache on the DataFrame before the write, so the write reads its data from memory and produces fewer files in each daily run: Caching changes where input data is served from, not how many output partitions the writer receives. It may consume memory without reducing the file count.

Raise spark.sql.shuffle.partitions for the job, so the final stage writes the 2 GB across more tasks and more evenly sized files in the table: More shuffle partitions generally create more tasks and can

produce still smaller output files if they reach the write stage. The immediate experiment aims to reduce writer partitions.

References:

1. Control data file size | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/tables/tune-file-size>

Question 35

An engineer adds a file arrival trigger to a job, pointing it at a cloud storage path where vendor files land. Saving fails, although notebooks read the path through an instance profile.

What must be true of the monitored location?

- A. It must be a Unity Catalog volume or external location path, which the trigger then monitors using Unity Catalog credentials
- B. It must be a workspace files folder under /Workspace, which the trigger then monitors using the job owner's workspace permissions
- C. It must be a DBFS mount of the storage path, which the trigger then monitors using the instance profile of the job cluster
- D. It must be a path holding fewer than ten files, which the trigger then monitors using the instance profile of the job cluster

Correct answer: A

A. It must be a Unity Catalog volume or external location path, which the trigger then monitors using Unity Catalog credentials

Explanation

Correct answer:

It must be a Unity Catalog volume or external location path, which the trigger then monitors using Unity Catalog credentials

Why this is correct:

Lakeflow Jobs file-arrival triggers monitor supported Unity Catalog paths, such as a volume or an external location, and use Unity Catalog credentials for access. An instance profile that lets a notebook read arbitrary storage does not make that path eligible for the trigger. The monitored path must be represented through the supported UC storage governance layer.

Why the other options are wrong:

It must be a workspace files folder under /Workspace, which the trigger then monitors using the job owner's workspace permissions: Workspace files are for workspace artifacts, not the supported cloud-storage locations for file-arrival monitoring. The trigger requires a Unity Catalog volume or external location path.

It must be a DBFS mount of the storage path, which the trigger then monitors using the instance profile of the job cluster: The instance profile controls cluster access, whereas the file-arrival trigger needs a supported Unity Catalog path and credentials. A DBFS mount does not satisfy that requirement.

It must be a path holding fewer than ten files, which the trigger then monitors using the instance

profile of the job cluster: The trigger requirement is about the path's Unity Catalog representation and access, not a low file-count threshold. Reducing the number of files would not fix an unsupported location.

References:

1. Trigger jobs when new files arrive | Databricks on AWS:

<https://docs.databricks.com/aws/en/jobs/file-arrival-triggers>

Question 36

Three days ago, the owner converted an external table to managed with the statement below. It must now be reversed; the original external location exists and rollback prerequisites are met.

```
ALTER TABLE corp.raw.measurements SET MANAGED;
```

Which statement reverses the conversion?

- A. DROP TABLE corp.raw.measurements, then CREATE TABLE with LOCATION, which registers the table again at its original external path
- B. ALTER TABLE corp.raw.measurements SET EXTERNAL, which is supported within 14 days of the conversion and restores its original path
- C. RESTORE TABLE corp.raw.measurements TO VERSION AS OF 0, which returns the table to its external location as it was before conversion
- D. ALTER TABLE corp.raw.measurements UNSET MANAGED, which is supported within 14 days of the conversion and reverts it back to external form

Correct answer: D

D. ALTER TABLE corp.raw.measurements UNSET MANAGED, which is supported within 14 days of the conversion and reverts it back to external form

Explanation

Correct answer:

ALTER TABLE corp.raw.measurements UNSET MANAGED, which is supported within 14 days of the conversion and reverts it back to external form

Why this is correct:

Databricks supports reversing a recent managed conversion with ALTER TABLE ... UNSET MANAGED within the documented 14-day window, subject to rollback prerequisites. At three days, with the original external location available, this is the purpose-built reversal that preserves the table rather than dropping and recreating it.

Why the other options are wrong:

ALTER TABLE corp.raw.measurements SET EXTERNAL, which is supported within 14 days of the conversion and restores its original path: SET EXTERNAL is not the documented inverse syntax for this conversion. The supported operation removes the managed designation with UNSET MANAGED.

RESTORE TABLE corp.raw.measurements TO VERSION AS OF 0, which returns the table to its external location as it was before conversion: Delta RESTORE moves table data to an earlier version; it does not reverse the table's managed/external registration mode. The conversion rollback has its own DDL operation.

DROP TABLE corp.raw.measurements, then CREATE TABLE with LOCATION, which registers the table again at its original external path: Dropping and recreating risks losing registration or data and is not the supported rollback path. It also changes the table object instead of reversing the conversion in place.

References:

1. ALTER TABLE | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-ddl-alter-table>

Question 37

A bronze **STRING** column **qty** uses 'N/A' for missing values. Silver needs **qty** as **INT** with **NULL** for the placeholder, and ANSI mode makes casting 'N/A' directly raise an error.

```
SELECT CAST(_____ AS INT) AS qty FROM bronze_orders
```

Which expression completes the cast?

- A. **COALESCE**(qty, 'N/A'), which replaces the placeholder with **NULL** before the cast, so every remaining value converts to an integer
- B. **NULLIF**(qty, 'N/A'), which returns **NULL** whenever **qty** equals the placeholder, so the cast then receives **NULL** instead of that text
- C. **IFNULL**(qty, **NULL**), which checks **qty** for the placeholder text and swaps in **NULL**, leaving every other value for the cast to convert
- D. **NVL**(qty, '0'), which replaces the placeholder with zero before the cast, so the silver column keeps a numeric value in every row

Correct answer: B

B. **NULLIF**(qty, 'N/A'), which returns **NULL** whenever **qty** equals the placeholder, so the cast then receives **NULL** instead of that text

Explanation

Correct answer:

NULLIF(qty, 'N/A'), which returns **NULL** whenever **qty** equals the placeholder, so the cast then receives **NULL** instead of that text

Why this is correct:

NULLIF(qty, 'N/A') returns SQL **NULL** when the input equals the placeholder and otherwise returns the original string. Casting that result to **INT** converts valid quantities while avoiding the ANSI-mode error on N/A; a SQL **NULL** safely remains **NULL**. The transformation replaces only the defined sentinel.

Why the other options are wrong:

IFNULL(qty, **NULL**), which checks **qty** for the placeholder text and swaps in **NULL**, leaving every other value for the cast to convert: This expression tests whether **qty** is already SQL **NULL**, not whether it equals the text N/A. The malformed placeholder would still reach the cast.

COALESCE(qty, 'N/A'), which replaces the placeholder with **NULL** before the cast, so every remaining value converts to an integer: **COALESCE** returns the first non-**NULL** argument, so a non-**NULL** 'N/A' remains unchanged. It does not map that text placeholder to **NULL**.

NVL(qty, '0'), which replaces the placeholder with zero before the cast, so the silver column keeps

a numeric value in every row: NVL also checks for SQL NULL rather than the literal placeholder. Even if used to fill actual NULLs with zero, it would leave N/A to fail the integer cast.

References:

1. nullif function | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/functions/nullif>

Question 38

The experimenters group was granted **ALL PRIVILEGES** on the schema **corp.sandbox**. That grant must be removed, while a separate **SELECT** grant the group holds on one table in **corp.finance** must stay in place.

Which statement fits?

- A. **REVOKE SELECT ON SCHEMA corp.sandbox FROM experimenters**, which removes the whole grant, as **SELECT** is the one data privilege it carries
- B. **REVOKE USE SCHEMA ON SCHEMA corp.sandbox FROM experimenters**, which removes the grant along with each of the privileges it includes
- C. **REVOKE ALL PRIVILEGES ON SCHEMA corp.sandbox FROM experimenters**, which removes that grant and each privilege it had implied on the schema
- D. **REVOKE ALL PRIVILEGES ON CATALOG corp FROM experimenters**, which removes the schema grant along with each grant made on the catalog

Correct answer: C

C. **REVOKE ALL PRIVILEGES ON SCHEMA corp.sandbox FROM experimenters**, which removes that grant and each privilege it had implied on the schema

Explanation

Correct answer:

REVOKE ALL PRIVILEGES ON SCHEMA corp.sandbox FROM experimenters, which removes that grant and each privilege it had implied on the schema

Why this is correct:

The grant to remove is scoped to **corp.sandbox**, so revoke the privileges on that schema. **ALL PRIVILEGES** removes the schema-level grant's effective privileges, including those implied there, while leaving the unrelated table-level **SELECT** grant in **corp.finance** untouched. Revocation does not delete grants on other objects.

Why the other options are wrong:

REVOKE ALL PRIVILEGES ON CATALOG corp FROM experimenters, which removes the schema grant along with each grant made on the catalog: Revoking at the catalog scope targets a different object and can affect privileges granted on that catalog. It does not precisely remove only the schema grant described.

REVOKE SELECT ON SCHEMA corp.sandbox FROM experimenters, which removes the whole grant, as **SELECT** is the one data privilege it carries: This revokes just **SELECT** at the schema scope, not the full set of privileges granted with **ALL PRIVILEGES**. Other explicitly granted schema privileges could remain.

REVOKE USE SCHEMA ON SCHEMA corp.sandbox FROM experimenters, which removes the grant along with each of the privileges it includes: USE SCHEMA is only one privilege, and revoking it does not revoke the rest of the schema's privileges. The target grant must be removed at its actual scope.

References:

1. Manage privileges in Unity Catalog:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/manage-privileges/?language=SQL>

Question 39

A data steward must attach an ABAC column mask policy to the catalog corp. She holds USE CATALOG and EXECUTE on the mask function, and the compute supports ABAC, but CREATE POLICY fails with a permission error.

What does she additionally need?

- A. APPLY TAG on the catalog corp, as a policy attaches through the governed tags that she must be able to apply on that catalog
- B. MANAGE on the catalog corp or ownership of it, since creating a policy on any securable requires one of those two privileges
- C. SELECT on every table in the catalog, since a policy can mask columns only in the tables that its creator is able to read
- D. ASSIGN on each governed tag that the policy matches, as a policy can reference only the tags that its creator is able to assign

Correct answer: B

B. MANAGE on the catalog corp or ownership of it, since creating a policy on any securable requires one of those two privileges

Explanation

Correct answer:

MANAGE on the catalog corp or ownership of it, since creating a policy on any securable requires one of those two privileges

Why this is correct:

Creating an ABAC policy on a securable is an administrative action. The steward already has catalog usage and permission to execute the mask function, but still needs MANAGE on the catalog or ownership to create the policy there. Function access and policy-management authority address separate requirements.

Why the other options are wrong:

APPLY TAG on the catalog corp, as a policy attaches through the governed tags that she must be able to apply on that catalog: Applying tags to an object is not the same permission as creating an ABAC policy on its catalog. APPLY TAG does not grant the administrative authority needed for the policy operation.

ASSIGN on each governed tag that the policy matches, as a policy can reference only the tags that its creator is able to assign: The ASSIGN privilege governs applying a tag, not creating an ABAC policy. A tag-based rule may refer to tags without requiring the policy creator to hold ASSIGN on every tag.

SELECT on every table in the catalog, since a policy can mask columns only in the tables that its creator is able to read: SELECT controls reading table data, not managing policy definitions. A policy author does not need broad read access to every table in the catalog just to attach a policy.

References:

1. Create and manage row filter and column mask policies | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/data-governance/unity-catalog/abac/policies>

Question 40

A vendor corrects delivered files by overwriting them in place under the same path. A new Auto Loader stream must pick up these corrected versions, and a downstream MERGE already handles any repeated business keys.

Which reader option should be set?

- A. `cloudFiles.useNotifications` set to true, so the storage events raised by each overwrite cause Auto Loader to reprocess that same file
- B. `cloudFiles.includeExistingFiles` set to true, so the stream rereads every file in the directory whenever any one of those files changes
- C. `cloudFiles.schemaEvolutionMode` set to rescue, so the changed contents of each overwritten file land in the rescued data column instead
- D. `cloudFiles.allowOverwrites` set to true, so a file replaced in place is picked up again and processed as new input data by the stream

Correct answer: D

D. `cloudFiles.allowOverwrites` set to true, so a file replaced in place is picked up again and processed as new input data by the stream

Explanation

Correct answer:

`cloudFiles.allowOverwrites` set to true, so a file replaced in place is picked up again and processed as new input data by the stream

Why this is correct:

Auto Loader normally treats discovered files as processed once, so a corrected object at an already-seen path may otherwise be ignored. Setting `cloudFiles.allowOverwrites=true` enables reprocessing when a file is overwritten in place. Since the downstream MERGE handles repeated business keys, the stream can safely pass the corrected file through again.

Why the other options are wrong:

`cloudFiles.useNotifications` set to true, so the storage events raised by each overwrite cause Auto Loader to reprocess that same file: Notifications affect how Auto Loader discovers new files; they do not by themselves change the handling of a path already considered processed. The overwrite option controls reprocessing behavior.

`cloudFiles.schemaEvolutionMode` set to rescue, so the changed contents of each overwritten file land in the rescued data column instead: Schema evolution handles structural changes to input records, not replacement of file contents at an existing path. The corrected file may have the same schema and still require overwrite processing.

cloudFiles.includeExistingFiles set to true, so the stream rereads every file in the directory whenever any one of those files changes: includeExistingFiles governs files present when a stream starts, not a rescan of the directory after every overwrite. It does not establish ongoing reprocessing of a previously seen path.

References:

1. Auto Loader FAQ | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/ingestion/cloud-object-storage/auto-loader/faq>

Question 41

An engineer must remove the unused column `legacy_flag` from a large Delta table without rewriting data files. The first attempt fails because the table does not support dropping columns.

```
ALTER TABLE corp.silver.orders DROP COLUMN legacy_flag;
```

What should be done before retrying?

- A. Set the table property `delta.columnMapping.mode` to `name`, which lets Delta drop the column as a metadata-only change to the table
- B. Run `ALTER COLUMN legacy_flag DROP NOT NULL` first, which releases the column's constraint so that it can then be dropped
- C. Set the table property `delta.enableChangeDataFeed` to `true`, which records the schema change and allows the column to be dropped
- D. Run `OPTIMIZE` on the table first, which rewrites its data files so that the column can then be dropped as a metadata-only change

Correct answer: A

- A. Set the table property `delta.columnMapping.mode` to `name`, which lets Delta drop the column as a metadata-only change to the table

Explanation

Correct answer:

Set the table property `delta.columnMapping.mode` to `name`, which lets Delta drop the column as a metadata-only change to the table

Why this is correct:

Delta column mapping lets a table track logical column names separately from physical file column names. With mapping mode set to `name`, dropping a column can be recorded as a metadata change rather than rewriting every Parquet file. This meets the requirement to remove the field without rewriting the large table's data files.

Why the other options are wrong:

Run `ALTER COLUMN legacy_flag DROP NOT NULL` first, which releases the column's constraint so that it can then be dropped: Nullability is a column constraint and has no bearing on whether Delta supports metadata-only column removal. Relaxing it would not enable the needed table feature.

Run `OPTIMIZE` on the table first, which rewrites its data files so that the column can then be dropped as a metadata-only change: Compacting data files does not enable column mapping. Rewriting files beforehand defeats the no-rewrite requirement and still does not correct the missing table capability.

Set the table property `delta.enableChangeDataFeed` to `true`, which records the schema change and allows the column to be dropped: Change Data Feed records row changes for downstream consumers; it does not authorize metadata-only column drops. Enabling it does not replace column mapping.

References:

1. Rename and drop columns with Delta Lake column mapping:

<https://docs.databricks.com/gcp/en/tables/features/column-mapping>

Question 42

Before a vendor feed's first load, an engineer wants to spot-check that the first rows parse against the table's schema without writing rows or recording files as loaded.

```
COPY INTO corp.raw.invoices  
FROM '/Volumes/corp/landing/invoices/'  
FILEFORMAT = PARQUET
```

Which clause completes the statement?

- A. COPY_OPTIONS ('mergeSchema' = 'false'), which checks the files against the table schema and rolls the load back if any row fails the check
- B. COPY_OPTIONS ('dryRun' = 'true'), which parses every file against the table schema and returns the result without writing to the table
- C. COPY_OPTIONS ('force' = 'false'), which checks the files against the table schema and records them as loaded without writing any rows
- D. VALIDATE 100 ROWS, which parses the first 100 rows, checks them against the table schema, and returns the result without loading data

Correct answer: D

D. VALIDATE 100 ROWS, which parses the first 100 rows, checks them against the table schema, and returns the result without loading data

Explanation

Correct answer:

VALIDATE 100 ROWS, which parses the first 100 rows, checks them against the table schema, and returns the result without loading data

Why this is correct:

COPY INTO ... VALIDATE 100 ROWS checks a sample of source rows against the target schema and reports validation results without loading those rows. That is the right preflight when the goal is to inspect parse/schema compatibility but leave both the table and the file-ingestion tracking state untouched.

Why the other options are wrong:

COPY_OPTIONS ('dryRun' = 'true'), which parses every file against the table schema and returns the result without writing to the table: dryRun is not the documented COPY INTO validation clause shown here, and this wording claims validation of every file. Use VALIDATE with a row count for a bounded sample.

COPY_OPTIONS ('force' = 'false'), which checks the files against the table schema and records

them as loaded without writing any rows: force=false is not a schema-only preview. COPY INTO's file tracking is designed to avoid reloading files already ingested, not to validate them without writing rows.

COPY_OPTIONS ('mergeSchema' = 'false'), which checks the files against the table schema and rolls the load back if any row fails the check: mergeSchema concerns schema evolution behavior, not a rollback-based validation mode. A load that fails partway through is not the requested read-only sample check.

References:

1. COPY INTO | Databricks on AWS:

<https://docs.databricks.com/aws/en/sql/language-manual/delta-copy-into>

Question 43

A developer finished testing a feature with a bundle deployed to the dev target. The jobs, pipeline, and uploaded files from that deployment must be removed from the workspace, while the project files stay local.

Which command fits?

- A. databricks bundle destroy -t dev, which deletes the resources and artifacts that the bundle had previously deployed to that target
- B. databricks workspace delete on the bundle's root_path, which removes the uploaded files and with them the deployed jobs and pipeline
- C. databricks bundle deploy -t dev after removing each resource from databricks.yml, which deletes the removed resources from dev
- D. databricks bundle deployment unbind for each resource, followed by deleting the unbound jobs and pipeline by hand in the workspace

Correct answer: A

A. databricks bundle destroy -t dev, which deletes the resources and artifacts that the bundle had previously deployed to that target

Explanation

Correct answer:

databricks bundle destroy -t dev, which deletes the resources and artifacts that the bundle had previously deployed to that target

Why this is correct:

databricks bundle destroy -t dev removes workspace resources and artifacts deployed by the bundle to the selected target. It cleans the deployed job, pipeline, and uploaded bundle files while leaving the developer's local project directory in place. Selecting the target ensures the cleanup applies to the dev deployment.

Why the other options are wrong:

databricks bundle deployment unbind for each resource, followed by deleting the unbound jobs and pipeline by hand in the workspace: Unbinding removes the bundle's association with a resource; it does not itself delete that resource. Manual deletion adds steps and risks leaving bundle artifacts behind.

databricks workspace delete on the bundle's root_path, which removes the uploaded files and with them the deployed jobs and pipeline: Deleting a workspace root path removes files there, but it does not necessarily delete the deployed jobs and pipeline that reference them. Bundle destroy tracks and deletes the deployed resources.

databricks bundle deploy -t dev after removing each resource from databricks.yml, which deletes the removed resources from dev: Redeploying after removing resource definitions does not provide the requested explicit teardown of all deployed resources and artifacts. The destroy command is designed for this lifecycle step.

References:

1. bundle command group | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/cli/bundle-commands>

Question 44

A row filter must return all rows to members of the account-level group `regional_managers` in any workspace running the query, including metastore-attached workspaces where the group is not assigned.

Which function should the filter use?

- A. `current_user() = 'regional_managers'`, which compares the calling user with the group's name and so matches each member of the group
- B. `is_account_group_member('regional_managers')`, which checks membership in the account-level group whichever workspace runs the query
- C. `is_member('regional_managers')`, which checks group membership as seen by the workspace that runs the query, so it covers each workspace
- D. `has_tag_value('group', 'regional_managers')`, which checks the calling user's group tag and so matches each member of that group

Correct answer: B

B. `is_account_group_member('regional_managers')`, which checks membership in the account-level group whichever workspace runs the query

Explanation

Correct answer:

`is_account_group_member('regional_managers')`, which checks membership in the account-level group whichever workspace runs the query

Why this is correct:

`is_account_group_member` evaluates whether the current user is a direct or indirect member of an account-level group. Its scope works across workspaces attached to the metastore, including when the group is not assigned as a workspace-local group. That matches the policy's need to recognize regional managers consistently wherever the query runs.

Why the other options are wrong:

`is_member('regional_managers')`, which checks group membership as seen by the workspace that runs the query, so it covers each workspace: Workspace-level membership checks can fail to recognize an account group that is not assigned to that workspace. The stem explicitly requires account-wide membership.

`has_tag_value('group', 'regional_managers')`, which checks the calling user's group tag and so matches each member of that group: `has_tag_value` checks tag metadata, not the caller's identity-group membership. It would need a tag on some object and would not answer whether the user belongs to the named group.

current_user() = 'regional_managers', which compares the calling user with the group's name and so matches each member of the group: current_user() returns the individual user identity, not the name of a group they belong to. Comparing those strings cannot test group membership.

References:

1. is_account_group_member function | Databricks on AWS:

https://docs.databricks.com/aws/pt/sql/language-manual/functions/is_account_group_member

Question 45

Compliance requires that managed tables in a new finance schema store their data in a dedicated bucket, separate from the metastore's default storage. An external location already covers that bucket.

How should the schema be created?

- A. CREATE SCHEMA with default settings, then add LOCATION to each CREATE TABLE statement, so its managed tables write to that bucket
- B. CREATE SCHEMA with default settings, then run SET MANAGED on each table, which moves the table's data into that external location
- C. CREATE SCHEMA with MANAGED LOCATION set to a path under that external location, so the schema's managed tables store their data there
- D. CREATE SCHEMA with default settings, then set the bucket path as a schema property, which the managed tables read when they are created

Correct answer: C

C. CREATE SCHEMA with MANAGED LOCATION set to a path under that external location, so the schema's managed tables store their data there

Explanation

Correct answer:

CREATE SCHEMA with MANAGED LOCATION set to a path under that external location, so the schema's managed tables store their data there

Why this is correct:

A schema-level managed location determines where managed tables in that schema store their data, subject to a governed external location. Setting MANAGED LOCATION during schema creation establishes the dedicated storage boundary for tables created there. Individual external table locations do not control managed table storage.

Why the other options are wrong:

CREATE SCHEMA with default settings, then add LOCATION to each CREATE TABLE statement, so its managed tables write to that bucket: LOCATION on each table is for external storage placement and would require repeating configuration per table. The schema's managed tables need a schema-level managed location.

CREATE SCHEMA with default settings, then set the bucket path as a schema property, which the managed tables read when they are created: Unity Catalog reads managed storage from the schema's MANAGED LOCATION setting, not from an arbitrary schema property. Use the DDL clause designed for that storage inheritance.

CREATE SCHEMA with default settings, then run **SET MANAGED** on each table, which moves the table's data into that external location: Changing each table's management mode is not how to assign the schema's managed storage location. It also alters individual table types rather than establishing the requested schema policy.

References:

1. What are schemas in Databricks? | Databricks on AWS:

<https://docs.databricks.com/aws/en/schemas>

Question 46

A column mask on corp.crm.accounts correctly redacts account_number for the support group. Support members hold USE CATALOG and USE SCHEMA, yet their table queries fail with a permission error.

Which change lets them query it with the mask still applied?

- A. Grant BROWSE on the catalog to the support group, which lets the group read the masked rows without holding a privilege on the table
- B. Change the mask function to return the redacted value for support, as a mask that returns a value also grants read access to the table
- C. Grant EXECUTE on the mask function to the support group, since running the mask function grants access to the table it protects
- D. Grant SELECT on the table to the support group, since the mask only changes values that the group is already allowed to read there

Correct answer: D

D. Grant SELECT on the table to the support group, since the mask only changes values that the group is already allowed to read there

Explanation

Correct answer:

Grant SELECT on the table to the support group, since the mask only changes values that the group is already allowed to read there

Why this is correct:

A column mask changes the value returned for a protected column when a user queries a table; it does not grant permission to read that table. Since support already has the container privileges, granting table-level SELECT supplies the missing read permission. The mask continues to govern the account number in the rows they are allowed to query.

Why the other options are wrong:

Change the mask function to return the redacted value for support, as a mask that returns a value also grants read access to the table: Mask logic controls the result of an authorized read; it does not create the underlying SELECT grant. Data masking and object authorization are separate controls.

Grant EXECUTE on the mask function to the support group, since running the mask function grants access to the table it protects: EXECUTE lets a principal invoke a function, but it does not authorize reading the table protected by the mask. The table's SELECT privilege is still required.

Grant BROWSE on the catalog to the support group, which lets the group read the masked rows

without holding a privilege on the table: BROWSE supports metadata discovery and access requests rather than reading table rows. It cannot replace SELECT, even when those rows would be masked.

References:

1. Unity Catalog privileges reference | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/access-control/privileges-reference>

2. Row filters and column masks | Databricks on AWS:

<https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks>

Question 47

A Kafka stream first ran with `startingOffsets` set to `earliest`. It is restarted with the same checkpoint after the operator changes `startingOffsets` to `latest` to skip a backlog. The subscription is unchanged, and all offsets are retained.

Where does the restarted query resume?

- A. From the latest offsets for new partitions only, while existing partitions restart at the earliest offsets that the brokers still retain
- B. From the earliest offsets on every partition, as the original `startingOffsets` value was stored in the checkpoint and is reused on restart
- C. From the latest offsets on every partition, as the changed `startingOffsets` value now takes precedence over the offsets in the checkpoint
- D. From the offsets recorded in its checkpoint, because `startingOffsets` applies only when a query starts without any checkpointed progress

Correct answer: D

D. From the offsets recorded in its checkpoint, because `startingOffsets` applies only when a query starts without any checkpointed progress

Explanation

Correct answer:

From the offsets recorded in its checkpoint, because `startingOffsets` applies only when a query starts without any checkpointed progress

Why this is correct:

For a query restarting from the same checkpoint, the checkpoint's committed and available offsets define progress. `startingOffsets` initializes a new query without checkpointed progress; changing it does not rewind or jump the existing query. Since the checkpoint and subscription are unchanged, processing resumes from the recorded offsets.

Why the other options are wrong:

From the latest offsets for new partitions only, while existing partitions restart at the earliest offsets that the brokers still retain: New partitions can use the configured starting position when they first appear, but existing partitions do not rewind to their earliest retained offsets on restart. The checkpoint preserves their progress.

From the earliest offsets on every partition, as the original `startingOffsets` value was stored in the checkpoint and is reused on restart: The checkpoint records progress, but it does not preserve the original `startingOffsets` setting as a restart instruction. That option matters only when the query has no prior checkpointed progress.

From the latest offsets on every partition, as the changed startingOffsets value now takes precedence over the offsets in the checkpoint: Changing the initial offset cannot override offsets already stored in the checkpoint. To start fresh from latest, the query would need a new checkpoint, which also changes its recovery history.

References:

1. Structured Streaming + Kafka Integration Guide (Kafka broker version 0.10.0 or higher):

<https://spark.apache.org/docs/3.5.6/structured-streaming-kafka-integration.html>

Question 48

A pipeline must load valid orders into silver and keep every invalid order, meaning a NULL order_id or a negative amount, in a separate quarantine table for review. No record may be lost from either table.

Which design fits?

- A. Apply the rule with expect on silver, which moves each invalid row into a quarantine table that the pipeline creates for that rule
- B. Apply the rule with expect_or_fail on silver, which stops the update and first writes the invalid rows to a separate quarantine table
- C. Define the rule once, apply it with expect_or_drop on silver, and build the quarantine table from rows where the rule is false
- D. Apply the rule with expect_or_drop on silver, and then read the dropped rows from the pipeline event log into the quarantine table

Correct answer: C

C. Define the rule once, apply it with expect_or_drop on silver, and build the quarantine table from rows where the rule is false

Explanation

Correct answer:

Define the rule once, apply it with expect_or_drop on silver, and build the quarantine table from rows where the rule is false

Why this is correct:

An expectation can define the validity rule once, while separate pipeline outputs preserve valid and invalid rows. Applying expect_or_drop to the silver output removes invalid rows there; a quarantine output derived from the same source applies the inverse predicate so those rows are retained separately. The event log records metrics, not the dropped row payloads.

Why the other options are wrong:

Apply the rule with expect_or_drop on silver, and then read the dropped rows from the pipeline event log into the quarantine table: Expectation event-log records provide counts and metrics rather than the discarded row contents needed for review. Once rows are dropped from that output, they must be routed from a data branch before being discarded.

Apply the rule with expect_or_fail on silver, which stops the update and first writes the invalid rows to a separate quarantine table: expect_or_fail fails the update when a row violates the rule; it does not automatically write those records to a quarantine table. Failure handling and data routing are separate designs.

Apply the rule with expect on silver, which moves each invalid row into a quarantine table that the pipeline creates for that rule: expect records metrics while retaining invalid rows in the output; it does not move them into a generated quarantine table. A separate branch and predicate are needed to isolate them.

References:

1. Expectation recommendations and advanced patterns | Databricks on AWS:

<https://docs.databricks.com/aws/en/ldp/expectation-patterns>

Question 49

A developer terminates the classic all-purpose cluster attached to a saved workspace notebook. The notebook still opens and can be edited before another cluster is attached, although its earlier variables and cached DataFrames are gone.

Which architecture detail explains this behavior?

- A. The notebook is written to the workspace's DBFS root by the cluster, while its variables were held in the executor memory of each worker
- B. The notebook is kept on the driver's local disk, which survives termination, while its variables lived only in the driver's memory
- C. The notebook is saved as a Unity Catalog table in the workspace catalog, while its variables lived only in the cluster's local cache
- D. The notebook is a workspace object kept by the control plane, while its variables and cached data lived only on the terminated compute

Correct answer: D

D. The notebook is a workspace object kept by the control plane, while its variables and cached data lived only on the terminated compute

Explanation

Correct answer:

The notebook is a workspace object kept by the control plane, while its variables and cached data lived only on the terminated compute

Why this is correct:

Workspace objects such as notebooks are managed by the Databricks control plane, while the cluster supplies the compute environment for running code. Terminating that compute removes its process memory and cached data, but does not delete the separately saved notebook object. This distinction explains why the file remains editable while runtime state disappears.

Why the other options are wrong:

The notebook is kept on the driver's local disk, which survives termination, while its variables lived only in the driver's memory: The driver's local disk is part of the compute environment and does not provide durable workspace storage after cluster termination. Notebook persistence is managed separately from that disk.

The notebook is saved as a Unity Catalog table in the workspace catalog, while its variables lived only in the cluster's local cache: Unity Catalog tables store data, whereas notebooks are workspace objects. Treating the notebook as table data confuses two different object types.

The notebook is written to the workspace's DBFS root by the cluster, while its variables were held in the executor memory of each worker: DBFS root is not where a classic cluster must persist saved workspace notebooks. Workspace storage keeps the notebook independently, while executors and their memory vanish with compute.

References:

1. High-level architecture | Databricks on AWS:

<https://docs.databricks.com/aws/en/getting-started/high-level-architecture>

Question 50

A bundle job needs the ID of the SQL warehouse named Shared Analytics, which differs in each workspace. The engineer wants the bundle to resolve the ID from that name at deploy time instead of hard-coding one ID per target.

Which variable definition fits?

A. `variables: {warehouse_id: {type: warehouse, default: "Shared Analytics"}}`

B. `variables: {warehouse_id: {lookup: {warehouse: "Shared Analytics"}}}`

C. `variables: {warehouse_id: {lookup: {name: "Shared Analytics"}}}`

D. `variables: {warehouse_id: {default: {warehouse: "Shared Analytics"}}}`

Correct answer: B

B. `variables: {warehouse_id: {lookup: {warehouse: "Shared Analytics"}}}`

Explanation

Correct answer:

`variables: {warehouse_id: {lookup: {warehouse: "Shared Analytics"}}}`

Why this is correct:

Bundle variable lookup supports resolving a named warehouse to its workspace-specific ID at deployment time. Defining `warehouse_id` with `lookup: {warehouse: "Shared Analytics"}` lets each workspace resolve the matching object rather than requiring a hard-coded ID per target. The resulting variable can then populate the job's warehouse ID field.

Why the other options are wrong:

variables: `{warehouse_id: {lookup: {name: "Shared Analytics"}}}`: The lookup key names the supported object type, not a generic name field. Use `warehouse` to ask the bundle to resolve a SQL warehouse by its name.

variables: `{warehouse_id: {type: warehouse, default: "Shared Analytics"}}`: Declaring a variable type and default string does not perform a named-object lookup. It would supply the text `Shared Analytics`, not resolve its workspace-specific ID.

variables: `{warehouse_id: {default: {warehouse: "Shared Analytics"}}}`: A default value is a literal fallback, not an object-resolution instruction. The lookup mapping is what retrieves the ID from the current workspace.

References:

1. Substitutions and variables in Declarative Automation Bundles | Databricks on AWS:

<https://docs.databricks.com/aws/en/dev-tools/bundles/variables>

Question 51

A tuning change looks faster, but its test read fewer source rows on a warm cache, while the baseline read the full input on a cold cache. The team needs evidence that the setting itself improved performance.

What should be done next?

- A. Rerun both versions on the same full input from a cold cache, placing the new version on a larger cluster so that it finishes sooner
- B. Rerun both versions on the same full input, comparable compute, and matching cache state, then compare the two resulting timings
- C. Rerun both versions from a cold cache on comparable compute, keeping the smaller input for the new version to shorten each test
- D. Rerun both versions on the same full input and comparable compute, starting the new version warm and the baseline cold as before

Correct answer: B

B. Rerun both versions on the same full input, comparable compute, and matching cache state, then compare the two resulting timings

Explanation

Correct answer:

Rerun both versions on the same full input, comparable compute, and matching cache state, then compare the two resulting timings

Why this is correct:

A fair performance comparison holds workload and execution conditions steady so the setting under test is the main changed variable. Both runs need the full same input, comparable compute, and matching cache state; otherwise input volume, hardware, or cache warmth can explain the timing difference. Then the team can attribute a measured change more credibly to the setting.

Why the other options are wrong:

Rerun both versions on the same full input from a cold cache, placing the new version on a larger cluster so that it finishes sooner: Changing to a larger cluster for only one version confounds the comparison with additional compute. Even though input and cache are matched, the timing cannot isolate the setting's effect.

Rerun both versions on the same full input and comparable compute, starting the new version warm and the baseline cold as before: Warm cache for one version and cold cache for the other changes how much work storage must do. A timing difference under those conditions is not an apples-to-apples test.

Rerun both versions from a cold cache on comparable compute, keeping the smaller input for the new version to shorten each test: Different input volumes mean the two timings measure different workloads. A cold cache and comparable compute do not compensate for the missing rows.

References:

1. Diagnose Lakeflow Jobs performance | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/diagnose-job-performance>

Question 52

Another team's job updates the Unity Catalog table `corp.shared.customers` at varying times each day. A downstream job owned by a different team must start after each update without being wired into the other team's job.

Which trigger fits?

- A. A table update trigger on the customers table, which starts that job once the monitored table receives each new commit
- B. A Run Job task added to the other team's job, which starts the downstream job after each update that the other team makes
- C. A file arrival trigger on a volume where the other team's job writes a success marker file after each update to that table
- D. A scheduled trigger every 15 minutes, which starts the downstream job often enough to follow each update made to that table

Correct answer: A

A. A table update trigger on the customers table, which starts that job once the monitored table receives each new commit

Explanation

Correct answer:

A table update trigger on the customers table, which starts that job once the monitored table receives each new commit

Why this is correct:

A table update trigger watches commits to a Unity Catalog table and starts its configured downstream job when the table changes. It allows one team's job to remain independent from another team's orchestration while reacting to updates at varying times. This is more direct than polling on a fixed schedule.

Why the other options are wrong:

A scheduled trigger every 15 minutes, which starts the downstream job often enough to follow each update made to that table: A 15-minute schedule polls at intervals and may run unnecessarily or lag behind a commit. It does not start specifically in response to each table update.

A Run Job task added to the other team's job, which starts the downstream job after each update that the other team makes: An explicit task dependency would require changing the other team's orchestration and ownership. The requirement is to remain unwired from their job definition.

A file arrival trigger on a volume where the other team's job writes a success marker file after

each update to that table: A marker file requires the other team's workflow to create and coordinate an additional signal. The table update trigger can observe the committed UC table directly.

References:

1. Trigger jobs when source tables are updated | Databricks on Google Cloud:

<https://docs.databricks.com/gcp/en/jobs/trigger-table-update>

Question 53

A gold aggregate must reflect every update and delete made to its silver source, which a MERGE job rewrites each hour. Queries should read precomputed results, and the aggregate must refresh after each silver update.

Which gold object fits?

- A. A materialized view over the silver table, which recomputes its results on each refresh so updates and deletes in the source are reflected
- B. A standard view over the silver table, which stores its results after each MERGE and serves them to later queries without recomputing
- C. A streaming table over the silver table, which processes each changed row once, so updates and deletes flow through on every run
- D. A temporary view over the silver table, which is recreated after each MERGE and keeps precomputed results for later dashboard sessions

Correct answer: A

A. A materialized view over the silver table, which recomputes its results on each refresh so updates and deletes in the source are reflected

Explanation

Correct answer:

A materialized view over the silver table, which recomputes its results on each refresh so updates and deletes in the source are reflected

Why this is correct:

A materialized view stores the result of a query and is refreshed to reflect changes in its source, including updates and deletes. Databricks may use incremental refresh when possible or recompute more broadly as needed, but the view's refreshed result remains consistent with the current source. That gives dashboards precomputed aggregate results after each refresh.

Why the other options are wrong:

A temporary view over the silver table, which is recreated after each MERGE and keeps precomputed results for later dashboard sessions: A temporary view is a query definition, not a persistent precomputed result. Recreating it does not retain a materialized aggregate for later dashboard sessions.

A streaming table over the silver table, which processes each changed row once, so updates and deletes flow through on every run: This object processes an append-oriented stream of new data; it is not the fitting abstraction for keeping an aggregate aligned to arbitrary source updates and deletes.

A standard view over the silver table, which stores its results after each MERGE and serves them to later queries without recomputing: A standard view stores the query definition rather than its result. Queries evaluate the definition against current source data instead of reading a persisted aggregate snapshot.

References:

1. How do pipelines refresh? | Databricks on AWS:

<https://docs.databricks.com/aws/en/ldp/concepts/refresh>

Question 54

A partner keeps a curated product catalog as Delta tables in its own Databricks account on another cloud and will not issue workspace credentials. The team needs the latest version without copying files or running extraction jobs.

Which approach fits this need?

- A. Lakehouse Federation to the partner's workspace, with a connection and a foreign catalog, so the team's queries read its tables in place
- B. Auto Loader reading the partner's table directory with cross-cloud credentials, which processes each new data file incrementally
- C. A managed ingestion connector for the partner's account, which copies the shared tables into the team's catalog on a schedule
- D. Delta Sharing, where the partner shares the tables and the team creates a catalog from the share to query the latest data in place

Correct answer: D

D. Delta Sharing, where the partner shares the tables and the team creates a catalog from the share to query the latest data in place

Explanation

Correct answer:

Delta Sharing, where the partner shares the tables and the team creates a catalog from the share to query the latest data in place

Why this is correct:

Delta Sharing is designed to let a provider share live governed data with recipients, including across Databricks accounts and clouds. The recipient can create a catalog from the share and query the shared Delta tables without receiving the provider's workspace credentials or copying the files into a separate ingestion pipeline.

Why the other options are wrong:

Lakehouse Federation to the partner's workspace, with a connection and a foreign catalog, so the team's queries read its tables in place: Lakehouse Federation queries supported external database systems through connections; it is not the mechanism for accessing another Databricks account's Delta tables as a shared dataset. Sharing is the direct cross-account model here.

Auto Loader reading the partner's table directory with cross-cloud credentials, which processes each new data file incrementally: File discovery in cloud storage is a different capability from cross-account Delta table sharing. It would also require storage access credentials and would ingest copies rather than expose the provider's latest table data.

A managed ingestion connector for the partner's account, which copies the shared tables into the team's catalog on a schedule: Copying supported source data into the recipient environment is a different model from direct table sharing and requires ingestion operations. Delta Sharing exposes the provider's shared tables in place.

References:

- 1. What is the OpenSharing Databricks-to-Databricks protocol? | Databricks on AWS:**
<https://docs.databricks.com/aws/en/opensharing/share-data-databricks>

Question 55

A `status_history` table holds many rows per `order_id`, each with a `status` and `updated_at` timestamp. A report needs each order's latest status from one aggregation.

```
SELECT order_id, _____ AS latest_status
FROM status_history
GROUP BY order_id
```

Which expression completes the query?

- A. `first(status)`, which returns each order's latest status, since the rows arrive sorted by `updated_at` within each `order_id` group
- B. `max(status)`, which returns the status taken from the row that holds the latest `updated_at` value for each `order_id` in the group
- C. `max_by(updated_at, status)`, which returns the status that belongs to the latest `updated_at` value found within each `order_id` group
- D. `max_by(status, updated_at)`, which returns the status taken from the row that holds the latest `updated_at` value for each `order_id`

Correct answer: D

D. `max_by(status, updated_at)`, which returns the status taken from the row that holds the latest `updated_at` value for each `order_id`

Explanation

Correct answer:

`max_by(status, updated_at)`, which returns the status taken from the row that holds the latest `updated_at` value for each `order_id`

Why this is correct:

`max_by(value, ordering)` returns the value associated with the maximum ordering expression in each group. Grouping by `order_id` and using `max_by(status, updated_at)` therefore selects the status from the row with the latest timestamp, without relying on physical row order.

Why the other options are wrong:

`first(status)`, which returns each order's latest status, since the rows arrive sorted by `updated_at` within each `order_id` group: Aggregate `first` does not infer latest from arrival order, and grouped rows are not guaranteed to arrive sorted by timestamp. Explicitly selecting by the ordering column is necessary.

`max_by(updated_at, status)`, which returns the status that belongs to the latest `updated_at` value

found within each order_id group: The first argument is the value to return and the second is the ordering expression. This reverses them and asks for the timestamp associated with the maximum status, not the status at the latest timestamp.

max(status), which returns the status taken from the row that holds the latest updated_at value for each order_id in the group: MAX(status) compares status values according to their value ordering, not by the timestamp of their source rows. It can select a lexically greatest status unrelated to recency.

References:

1. max_by aggregate function | Databricks on AWS:

https://docs.databricks.com/aws/en/sql/language-manual/functions/max_by